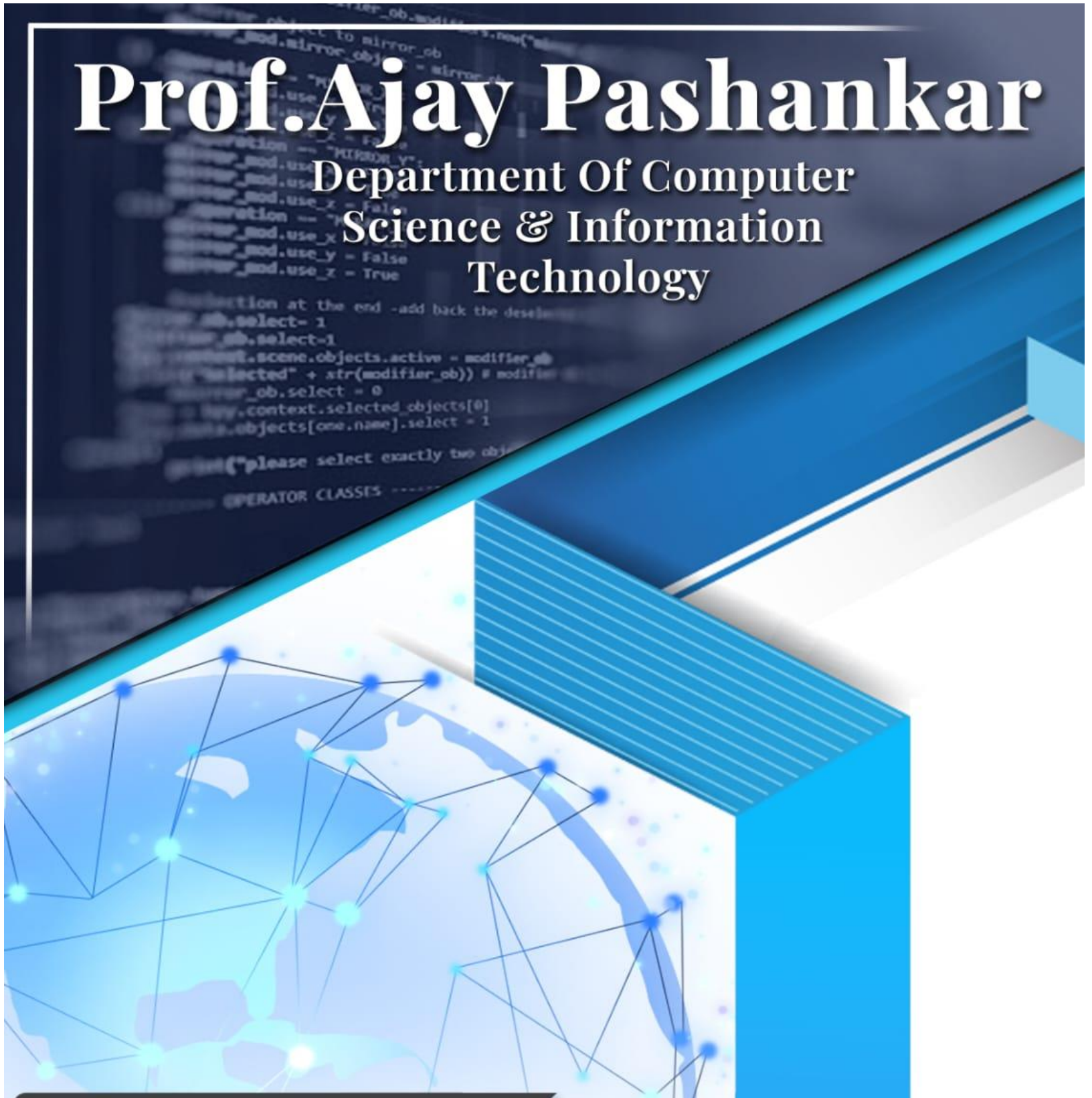
Prof.Ajay Pashankar

Department Of Computer Science & Information Technology



ajaypashankar7

Prof.Ajay Pashankar

www.profajaypashankar.com

TYIT FULL STACK DEVELOPMENT MERN SEM V BY: PROF. AJAY PASHANKAR

Sr. No.	Practical / Experiment	Date	Sign.
1(a)	Installation of Node.js and Visual Studio Code		
1(b)	Create a Simple Node.js File and Run it Using the Terminal		
1(c)	Write a Node.js Script Demonstrating Basic JavaScript Commands		
1(d)	Create and Run a Simple Program Using Variables and Functions		
1(e)	Demonstrate Use of Operators and Control Statements in JavaScript		
1(f)	Write Programs Using Loops and Template Literals		
2(a)	Create Functions and Call Them with Parameters		
2(b)	Work with Arrays and Objects in JavaScript		
2(c)	Demonstrate the Use of Arrow Functions		
2(d)	Write a Program Using Array Methods (map, filter, reduce)		
2(e)	Create a Program that Manipulates Objects and Displays Output		
2(f)	Implement a Small JavaScript Program Combining Functions, Arrays and Objects		
3(a)	Create a React Application Using Vite/CRA and Display 'Welcome to MERN Stack Development'		
3(b)	Display Dynamic Data Using JSX		
3(c)	Declare Variables and Write a Function to Calculate the Sum of Two Numbers		
3(d)	Apply Basic CSS Styling to a React Component		
3(e)	Create a Simple Functional Component and Render it in App.js		
3(f)	Create a React Component that Displays the Current Date and Time Dynamically		
4(a)	Create Multiple Components and Render Them on a Single Page		
4(b)	Pass Data Between Components Using Props		
4(c)	Use the useState Hook to Manage and Update Data		
4(d)	Create a Form Component to Accept User Input and Display it Dynamically		
4(e)	Display a List of Items Stored in the State Using map()		
4(f)	Build a Small Interactive Component (e.g., Counter or To-Do List)		
5(a)	Install and Configure React Router in the React Application		
5(b)	Create Multiple Pages and Implement Navigation Between Them		
5(c)	Use Link and Routes to Navigate Between Pages		
5(d)	Fetch Sample Data from a Public API Using Axios		
5(e)	Display Fetched Data in Table or List Format		
5(f)	Implement Search and Filter Functionality on Displayed Data		
6(a)	Install Express and Create a Simple Server		
6(b)	Create Multiple Routes in Express		
6(c)	Send HTML Content from an Express Server		
6(d)	Handle Route Parameters in Express		
6(e)	Handle Query Parameters in Express		

Sr. No.	Practical / Experiment	Date	Sign.
6(f)	Create a Simple JSON API Using Express		
7(a)	Install MongoDB Community Server and MongoDB Compass		
7(b)	Perform CRUD Operations Using MongoDB Compass		
7(c)	Connect Node.js with MongoDB Using Mongoose		
8(a)	Create a MongoDB Schema and Model Using Mongoose		
8(b)	Implement REST API Operations		
8(c)	Test REST APIs Using Postman		
9(a)	Create the React Frontend and Connect it with Express		
9(b)	Retrieve and Display Student Records		
9(c)	Add, Update and Delete Student Records		
10(a)	Design and Execute the Complete MERN Application		
10(b)	Implement Complete CRUD Operations		
10(c)	Test and Validate the Complete MERN Workflow		

Module 1: Frontend Development using React

1. Environment Setup and JavaScript Basics

a. Install Node.js and Visual Studio Code, and verify installation.

Requirements

- Node.js
- Visual Studio Code
- Command Prompt/Terminal
- Web Browser
- Basic knowledge of JavaScript

Node.js is a JavaScript runtime environment based on Chrome V8 engine.

NPM is the Node Package Manager used to manage packages.

Steps:

1. Download

Go to nodejs.org

Click on LTS version (it will show v24.x.x LTS)

2. Install (Windows)

Run the .msi file you downloaded

Next > Next > Accept

3. Verify

Open Command prompt/Terminal and verify the Node.js installation using:

node –v

Verify npm using:

npm –v

4. Install Visual Studio Code.

It is a lightweight code editor with an integrated terminal and extensions support for MERN development.

5. Open Visual Studio Code.

b. Create a simple Node.js file and run it using the terminal.

1. Create a folder named MERN_Practicals.

2. Open the folder in VS Code.

3. Create a file named app.js.

4. Add JavaScript code.

```
console.log("Hello Developers!");
```

console.log() - Used to print output on the terminal.

node fileName.js - Used to execute a JavaScript file.

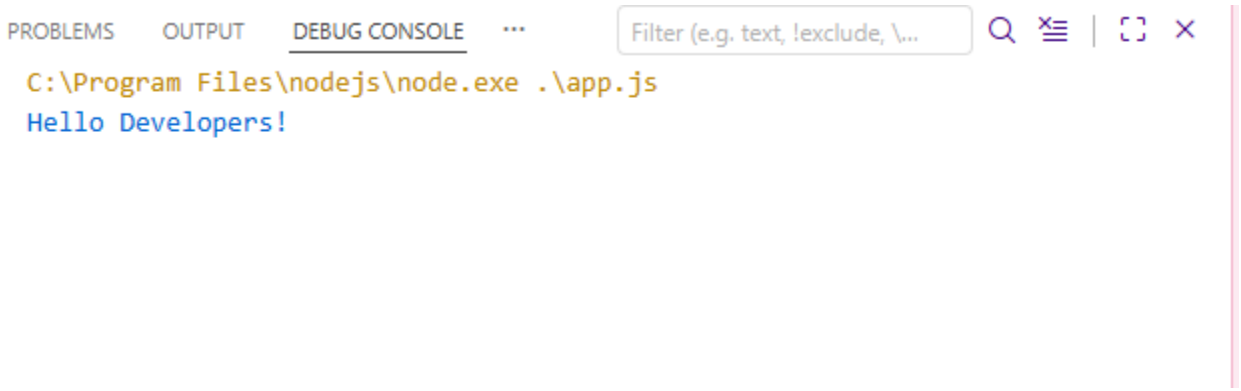
Key Point: Every Node.js file should have a .js extension.

5. Open the terminal.

6. Run the program using:

node.js

7. Output



c. Write a Node.js script demonstrating basic JavaScript commands (console log, variables, and arithmetic operations).

1. Create a file named app.js.

2. Add JavaScript code.

```
console.log("Hello, MERN Developer!");

let name = "jervy";
let age = 20;

console.log("Name:",name);
console.log("Age:",age);

let a = 10;
let b = 5;

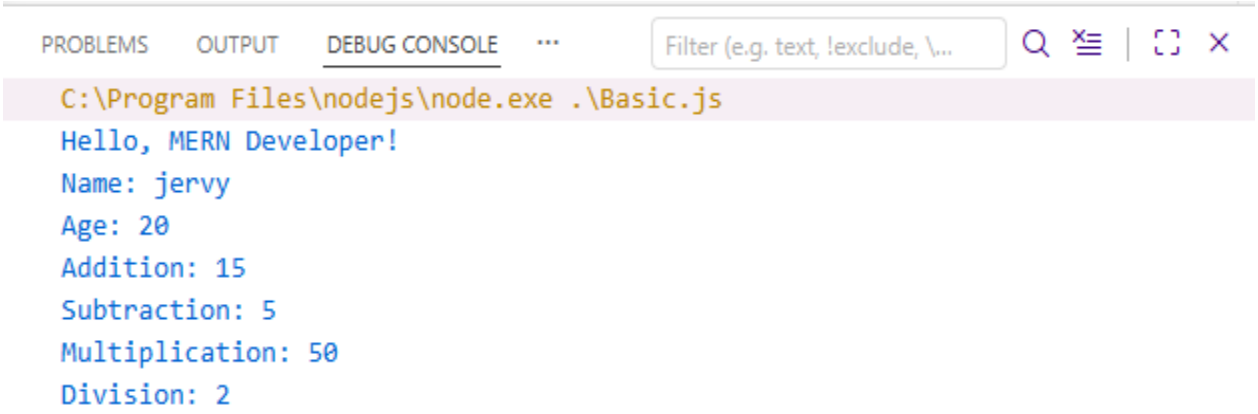
console.log("Addition:",a+b);
console.log("Subtraction:",a-b);
console.log("Multiplication:",a*b);
console.log("Division:",a/b);
```

3. Open the terminal.

4. Run the program using:

node.js

5. Output



d. Create and run a simple program using variables and functions.

1. Add JavaScript code.

```
let a = 20;
let b = 30;

function addNumbers(x, y) {
    return x + y;
}

let result = addNumbers(a, b);

console.log("First Number:", a);
console.log("Second Number:", b);
console.log("Sum:", result);
```

2. Open the terminal.

3. Run the program using:

node.js

4. Output



Variable Declaration: let, const, var

let can be reassigned, const cannot be reassigned

Key Point: A function is a reusable block of code designed to perform a particular task..

e. Demonstrate use of operators and control statements in JavaScript

1. Add JavaScript code.

```
let marks = 75;

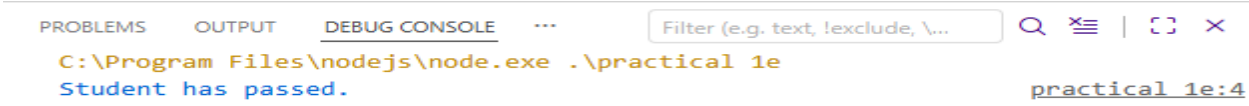
if (marks >= 40) {
    console.log("Student has passed.");
} else {
    console.log("Student has failed.");
}
```

2. Open the terminal.

3. Run the program using: node.js

4. Output

Key Point: A function is a reusable block of code designed to perform a particular task.



Important Operators:

Comparison: == (checks only value), === (checks value and data type)

Logical: && (AND), || (OR), ! (NOT)

Important Statements:

if, else if, else

switch statement

Ternary Operator:

condition ?

statement1 :

statement2

f. Write programs using loops and template literals.

1. Add JavaScript code.

```
let fruits = [
    "Apple",
    "Banana",
    "Cherry",
    "Date",
    "Elderberry"
];
for (let fruit of fruits) {
    console.log(`I like ${fruit}!`);
}
```

2. Open the terminal.

3. Run the program using:

node.js

4. Output



Important Loops:

- for loop, while loop, do...while loop
- for...of loop - used for arrays
- for...in loop - used for objects

Template Literals - a modern way to combine strings and variables

Key Point: Template literals use back-ticks and make string formatting easier and more readable than concatenation.

why Node.js is used?

- "Because it allows us to use JavaScript for both frontend and backend, so development becomes faster."

2. Core JavaScript Concepts for MERN.

a. Create functions and call them with parameters.

Requirements

- Visual Studio Code
- Node.js
- JavaScript
- Command Prompt / VS Code Terminal

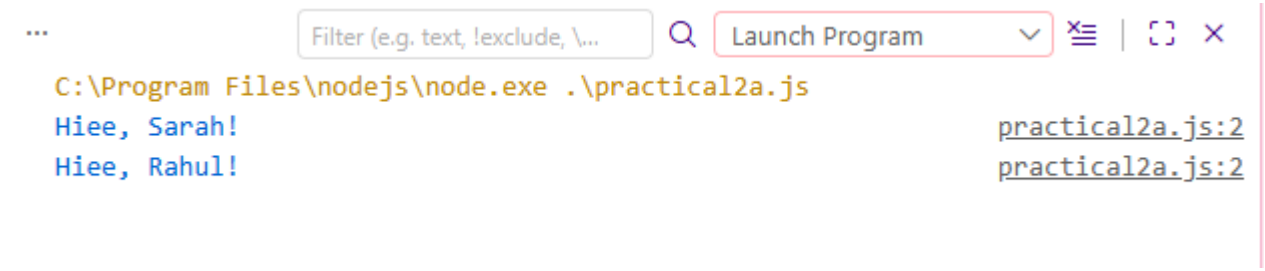
Steps

1. **Open Visual Studio Code.**
2. **Create a new file named practical2a.js.**
3. **Run the program using Node.js.**

```
function greet(name) {
    console.log(`Hiee, ${name}!`);
}

greet("Sarah");
greet("Rahul");
```

Output



b. Work with arrays and objects in JavaScript.

1. **Open Visual Studio Code.**
2. **Create a new file named practical2b.js.**
3. **Run the program using Node.js.**

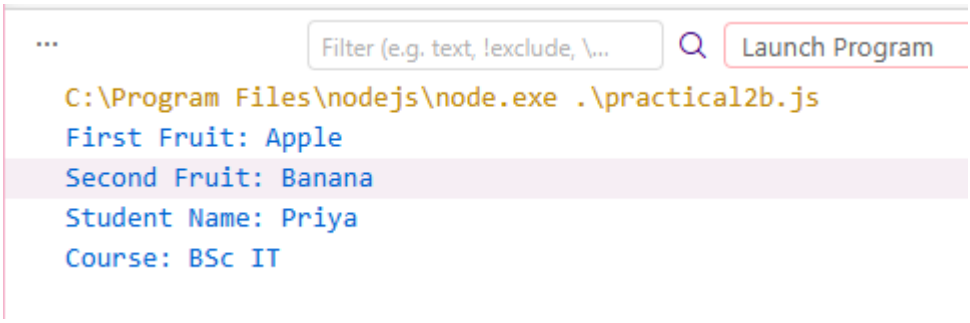
```
let fruits = ["Apple", "Banana", "Mango", "Orange"];

let student = {
    name: "Priya",
    age: 19,
    course: "BSc IT"
};

console.log("First Fruit:", fruits[0]);
console.log("Second Fruit:", fruits[1]);

console.log("Student Name:", student.name);
console.log("Course:", student.course);
```

Output



c. Demonstrate use of arrow functions.

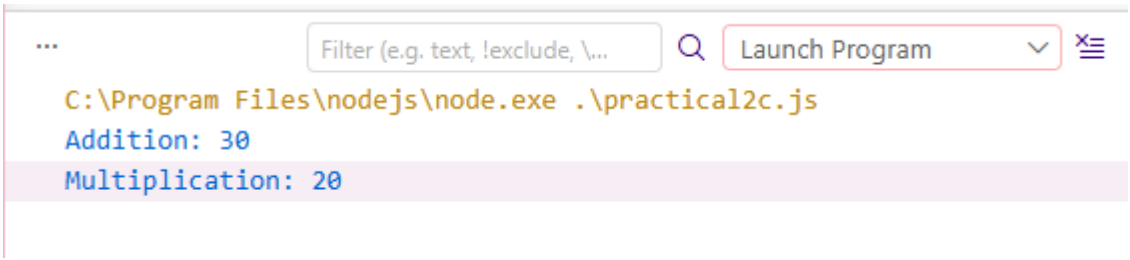
1. **Open Visual Studio Code.**
2. **Create a new file named practical2c.js.**
3. **Run the program using Node.js.**

```
const add = (a, b) => {
    return a + b;
};

const multiply = (a, b) => a * b;

console.log("Addition:", add(10, 20));
console.log("Multiplication:", multiply(5, 4));
```

Output



d. Write a program using array methods (map, filter, reduce).

1. **Open Visual Studio Code.**
2. **Create a new file named practical2d.js.**
3. **Run the program using Node.js.**

```
let numbers = [10, 20, 30, 40, 50];

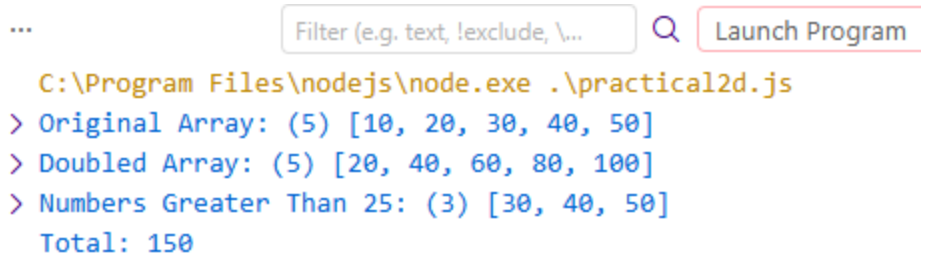
let doubled = numbers.map(num => num * 2);

let greaterThan25 = numbers.filter(num => num > 25);

let total = numbers.reduce((sum, num) => sum + num, 0);

console.log("Original Array:", numbers);
console.log("Doubled Array:", doubled);
console.log("Numbers Greater Than 25:", greaterThan25);
console.log("Total:", total);
```

Output



e. Create a program that manipulates objects and displays output.

1. **Open Visual Studio Code.**
2. **Create a new file named practical2e.js.**
3. **Run the program using Node.js.**

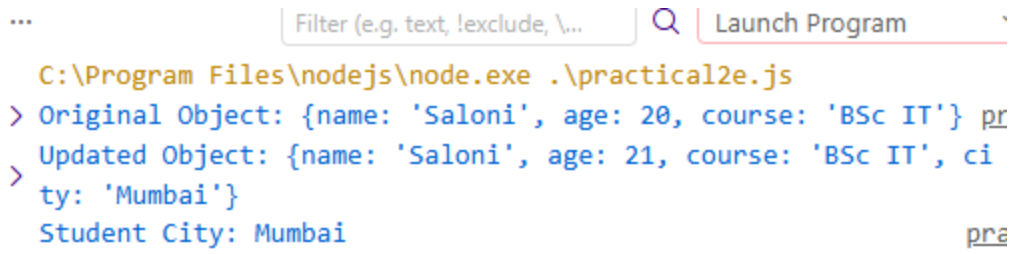
```
let student = {
    name: "Saloni",
    age: 20,
    course:"BSc IT"
};

console.log("Original Object:", student);

student.city = "Mumbai";
student.age = 21;

console.log("Updated Object:", student);
console.log("Student City:", student.city);
```

Output



f. Implement a small JavaScript program combining functions, arrays, and objects.

1. **Open Visual Studio Code.**
2. **Create a new file named practical2f.js.**
3. **Run the program using Node.js.**

```
let students = [
    {
        name: "isha",
        age: 20,
        course: "BSc IT"
    },
    {
        name: "Rahul",
        age: 21,
        course: "BCA"
    },
    {
        name: "Priya",
        age: 20,
        course: "BSc CS"
    }
];

function displayStudents(studentList) {
    for (let student of studentList) {
        console.log(`Name: ${student.name}`);
        console.log(`Age: ${student.age}`);
        console.log(`Course: ${student.course}`);
        console.log("--------------------");
    }
}

displayStudents(students);
```

Output

```
C:\Program Files\nodejs\node.exe .\practical2f.js
Name: isha
Age: 20
Course: BSc IT
--------------------
Name: Rahul
Age: 21
Course: BCA
--------------------
Name: Priya
Age: 20
Course: BSc CS
--------------------
```

Important Methods / Functions

1. function

Used to create a reusable block of JavaScript code.

2. console.log()

Used to display output in the console.

3. map()

Creates a new array by applying a function to every element.

4. filter()

Creates a new array containing elements that satisfy a condition.

5. reduce()

Reduces an array to a single value.

6. Arrow Function =>

Provides a shorter syntax for writing functions.

7. Object Property Access

object.property

Used to access a property of an object**.**

8. return

Returns a value from a function.

3. React Introduction and JSX

a. Create a React application using Vite/CRA and display "Welcome to MERN Stack Development".

Requirements

- Visual Studio Code
- Node.js and npm
- React
- Vite
- Web Browser

1. Open Command Prompt / VS Code Terminal

2. Run command:

npm create vite@latest my-mern-app -- --template react

Type y and press Enter

Run: cd my-mern-app

Run:

npm install - This installs all dependencies

```
C:\Users\Raj Salont\Desktop\MERN Practicals>npm create vite@latest
 my-react-app -- --template react

> npx
> create-vite my-react-app --template react

|
◇  Which linter to use?
|  Oxlint
|
◇  Install with npm and start now?
|  Yes
|
◇  Scaffolding project in C:\Users\Raj Salont\Desktop\MERN Practic
als\my-react-app...
|
◇  Installing dependencies with npm...

added 24 packages, and audited 25 packages in 28s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
◇  Starting dev server...

> my-react-app@0.0.0 dev
> vite


  VITE v8.2.2  ready in 1405 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
Terminate batch job (Y/N)? y

C:\Users\Raj Salont\Desktop\MERN Practicals>cd my-react-app

C:\Users\Raj Salont\Desktop\MERN Practicals\my-react-app>
```

3. Open this folder in VS Code

4. Go to src folder -> Open App.jsx

Delete all existing code and write this

```
function App() {
  return (
    <div>
      <h1>Welcome to MERN Stack Development</h1>
    </div>
  )
}
export default App
```

5. Run in the terminal:

npm run dev

Open link http://localhost:5173 in browser

Output



Error:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Raj Salont\Desktop\MERN Practicals> npm create vite@late
st my-react-app
npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded
because running scripts is disabled on this system. For more
information, see about_Execution_Policies at
https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ npm create vite@latest my-react-app
+ ~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityExc
   eption
    + FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\Raj Salont\Desktop\MERN Practicals>
```

Open command prompt and bash commands in it not in powershell

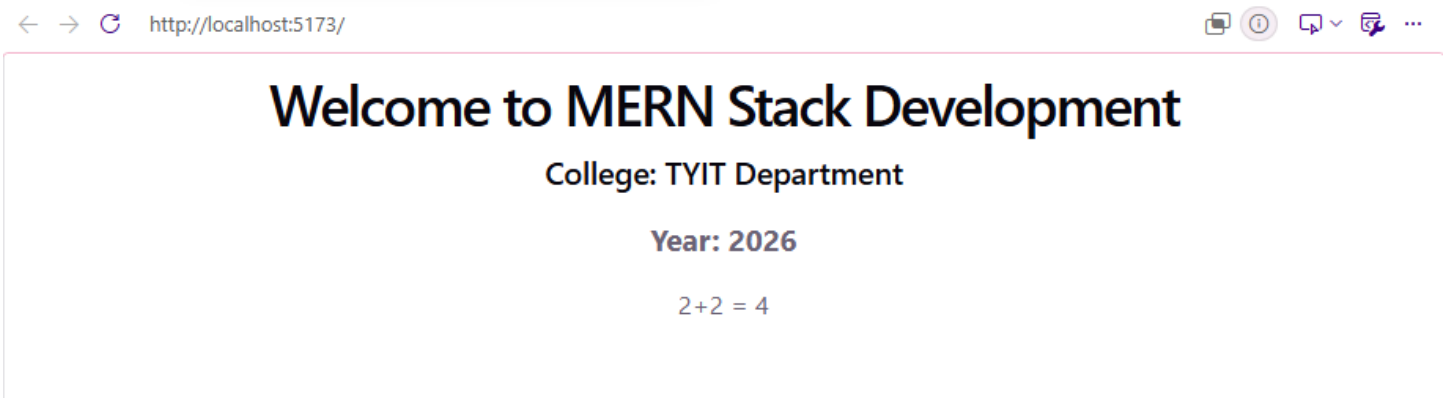
b. Display dynamic data using JSX.

Repeat the steps of previous practical

Now in App.jsx write:

```
function App() {
  const college = "TYIT Department";
  const year = 2026;
  return (
    <div>
      <h1>Welcome to MERN Stack Development</h1>
      <h2>College: {college}</h2>
      <h3>Year: {year}</h3>
      <p>2+2 = {2+2}</p>
    </div>
  );
}
export default App;
```

Output

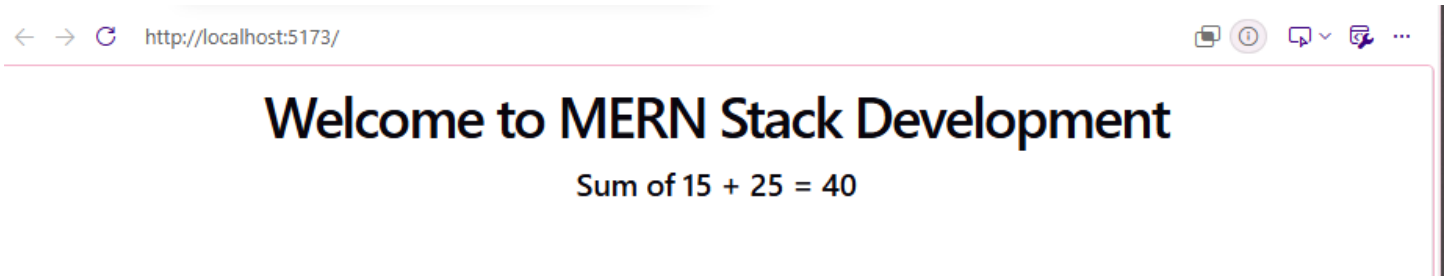


c. Declare variables and write a function to calculate the sum of two numbers and display the result.

In src/App.jsx

```
function App() {
  const a=15, b=25;
  function calculateSum(x,y){ return x+y; }
  return (
    <div>
      <h1>Welcome to MERN Stack Development</h1>
      <h2>Sum of {a} + {b} = {calculateSum(a,b)}</h2>
    </div>
  );
}
export default App;
```

OUtput



d. Apply basic CSS styling to a React component

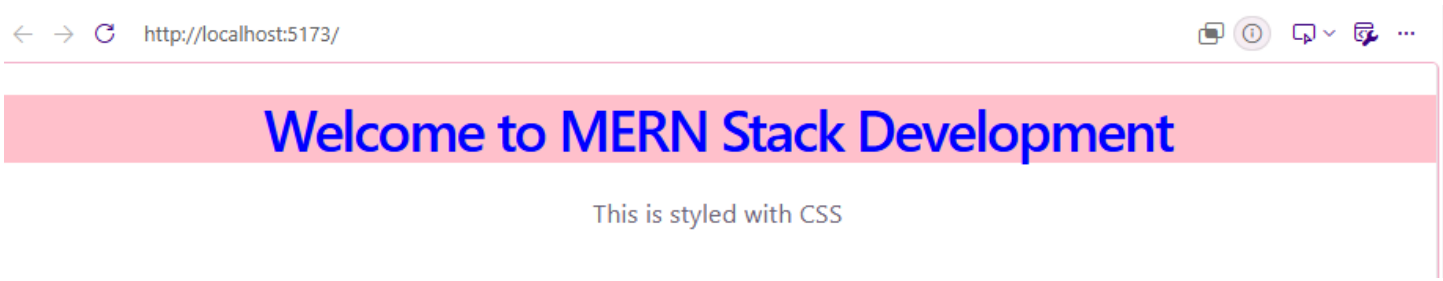
In src/App.jsx

```
import './App.css'
function App() {
  return (
    <div>
      <h1 style={{color:'blue', backgroundColor:'yellow', padding:'10px'}}>
        Welcome to MERN Stack Development
      </h1>
      <p className="myPara">This is styled with CSS</p>
    </div>
  );
}
export default App;
```

In App.css

```
.myPara{ color:green; font-size:22px; border:2px solid black; padding:10px; }
```

OUtput

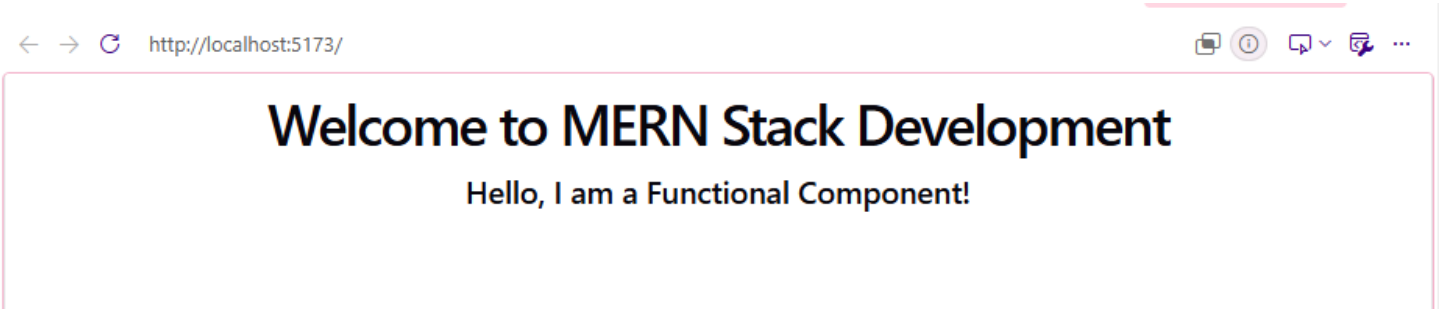


e. Create a simple functional component and render it in App.js.

In src/App.jsx

```
function Greeting(){
  return <h2>Hello, I am a Functional Component!</h2>
}
function App(){
  return (
    <div>
      <h1>Welcome to MERN Stack Development</h1>
      <Greeting />
    </div>
  );
}
export default App;
```

Output

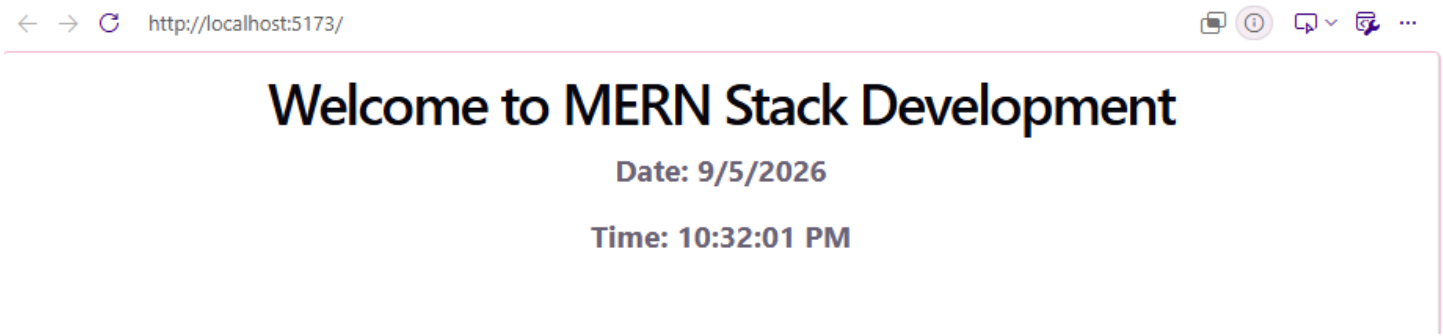


f. Create a React component that displays the current date and time dynamically using JavaScript.

In src/App.jsx

```
function App(){
  const date = new Date().toLocaleDateString();
  const time = new Date().toLocaleTimeString();
  return (
    <div>
      <h1>Welcome to MERN Stack Development</h1>
      <h3>Date: {date}</h3>
      <h3>Time: {time}</h3>
    </div>
  );
}
export default App;
```

Output



Important Methods / Functions

1. function App()

Used to create a reusable Functional Component in React.

2. return()

Used to return JSX, which is the HTML-like code to be displayed on screen.

3. export default

Used to export a component so it can be imported and used in other files.

4. JSX Expression { }

Used to embed and display JavaScript variables and expressions inside JSX.

5. import

Used to import CSS files and other components into the current file.

6. style={{ }}

Used to apply inline CSS styling to JSX elements in React.

7. className

Used to apply CSS classes from an external CSS file, instead of using `class`.

8. function call - calculateSum()

Used to create a reusable block of code and call it inside JSX to get a result.

9. new Date()

Used to create an object that contains the current date and time.

10. toLocaleDateString()

Used to display the current date in a readable string format**.**

11. toLocaleTimeString()

Used to display the current time in a readable string format.

12. Component Call <Greeting />

Used to call and display one functional component inside another component.

4. React Components, Props, and State

a. Create multiple components and render them on a single page.

Requirements

- VS Code
- Node.js
- React + Vite

1. Create a new React project using

npm create vite@latest practical-4 -- --template react

2. Go inside folder:

cd practical-4

3. install dependencies: npm install

Open src/App.jsx

write the required code

```
function Header(){ return <h1>Hey fellow developers</h1> }
function Footer(){ return <h1>May your coding journey be filled with
success!</h1> }
function App(){
  return <><Header /><Footer /></>
}
export default App;
```

4. Run the project:

npm run dev

Open browser link http://localhost:5173/ to see the output.

Hey fellow developers

May your coding journey be filled with success!

b. Pass data between components using props

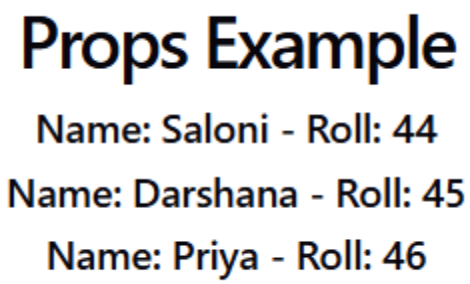
1. Open src/App.jsx

write the required code

```
// Child Component
function Student(props){
  return <h2>Name: {props.name} - Roll: {props.roll}</h2>
}

//Parent Component
function App(){
  return (
    <div>
      <h1>Props Example</h1>
      <Student name="Saloni" roll="44" />
      <Student name="Darshana" roll="45" />
      <Student name="Priya" roll="46" />
    </div>
  );
}
export default App;
```

Output



c. Use the useState hook to manage and update data.

1. Open src/App.jsx

write the required code

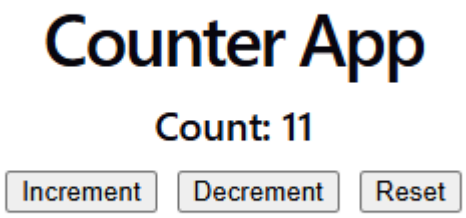
```
import { useState } from 'react'

function App() {
  const [count, setCount] = useState(0);

  return (
    <div style={{textAlign:'center', marginTop:'50px'}}>
      <h1>Counter App</h1>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count+1)}>Increment</button>
      <button onClick={() => setCount(count-1)}
style={{marginLeft:'10px'}}>Decrement</button>
      <button onClick={() => setCount(0)}
style={{marginLeft:'10px'}}>Reset</button>
    </div>
  );
}
export default App;
```

to run just click on the local host link in the terminal.

Output



d. Create a form component to accept user input and display it dynamically.

1. Open src/App.jsx

write the required code

```
import { useState } from 'react'

function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [course, setCourse] = useState("");

  return (
    <div style={{ textAlign: 'center', marginTop: '40px', fontFamily: 'Arial' }}>
      <h1>User Form</h1>

      <input
        type="text"
        placeholder="Enter Your Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
        style={{ padding: '8px', margin: '5px' }}
      />
      <br/>

      <input
        type="email"
        placeholder="Enter Your Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        style={{ padding: '8px', margin: '5px' }}
      />
      <br/>

      <input
        type="text"
        placeholder="Enter Your Course"
        value={course}
        onChange={(e) => setCourse(e.target.value)}
        style={{ padding: '8px', margin: '5px' }}
```

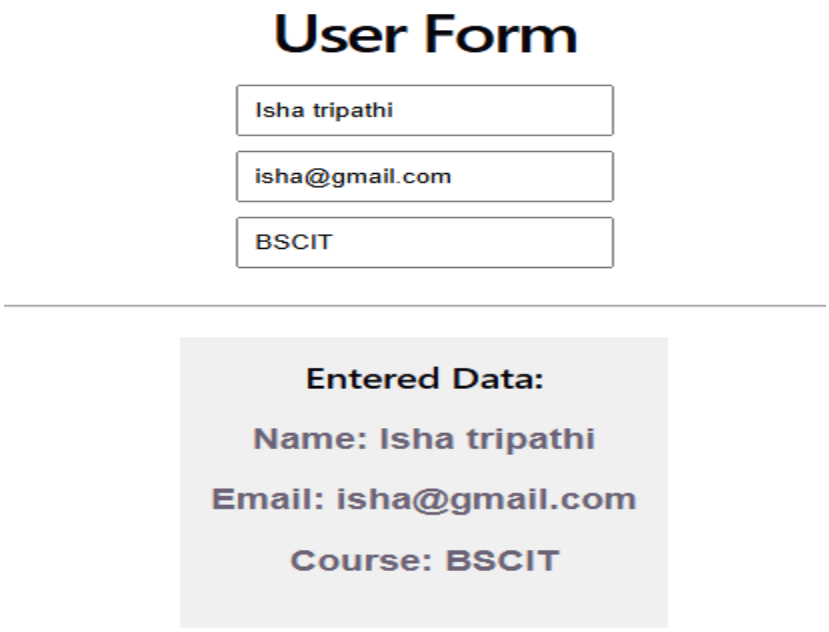
```
      />

      <hr style={{ margin: '20px' }} />

      {/* Dynamically Display */}
      <div style={{ backgroundColor: '#f0f0f0', padding: '15px', display:
'inline-block' }}>
        <h2>Entered Data:</h2>
        <h3>Name: {name}</h3>
        <h3>Email: {email}</h3>
        <h3>Course: {course}</h3>
      </div>
    </div>
  );
}

export default App;
```

Output



e. Display a list of items stored in the state using map().

1. Open src/App.jsx

write the required code

```
function App(){
  const list = ["Apple","Mango","Banana"];
  return <ul>{list.map((item,i)=><li key={i}>{item}</li>)}</ul>
}
export default App;
```

Output

- Apple
- Mango
- Banana

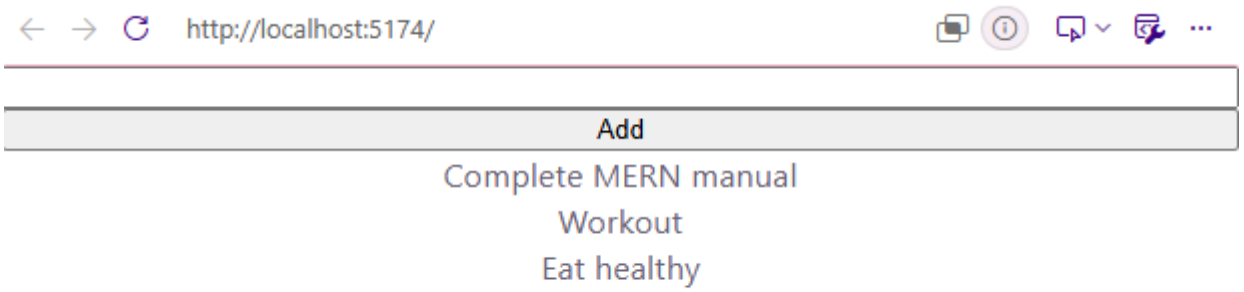
f. Build a small interactive component (e.g., counter or to-do list).

1. Open src/App.jsx

write the required code

```
import { useState } from 'react'
function App(){
  const [task, setTask] = useState("");
  const [tasks, setTasks] = useState([]);
  return <>
    <input value={task} onChange={e=>setTask(e.target.value)}/>
    <button onClick={()=>{setTasks([...tasks,task]); setTask("")}}>Add</button>
    {tasks.map((t,i)=><li key={i}>{t}</li>)}
  </>
}
export default App;
```

Output



Important Methods / Functions

1. `function App()`

Used to create a reusable Functional Component in React.

2. `return()`

Used to return JSX, which is the HTML-like code to be displayed on screen.

3. `useState()`

Used to create and manage state in a functional component. It returns a value and a function to update it.

4. `props`

Used to pass data from parent component to child component.

5. `setCount()` / `setName()`

Used to update the state value and re-render the component with new data.

6. `onClick={ }`

Used to handle click events on buttons and other elements in React.

7. `onChange={ }`

Used to handle input field changes and accept user input dynamically.

8. `map()`

Used to display a list of items stored in state or array by iterating over them.

9. `export default`

Used to export a component so it can be imported and used in other files.

5. React Routing and Data Handling

a. Install and configure React Router in the React application.

Requirements

- VS Code
- Node.js
- React application created using Vite

Create a React Application

Steps

1. **Open** VS Code**.**
2. **Open** Terminal → New Terminal**.**
3. **Create a React application:**

npm create vite@latest practical5 -- --template react

```
C:\Users\Raj Salont\Desktop\MERN Practicals\practical-5>npm install react-router

added 3 packages, and audited 28 packages in 9s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

4. Create page Components

Inside the Src folder, create a folder named:

Pages

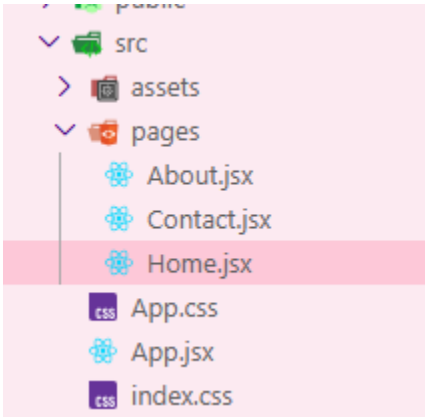
Inside it, create:

Home.jsx

About.jsx

Contact.jsx

It should be like this:



In Home.jsx

```
function Home() {

  return (
    <div>
      <h1>Home Page</h1>
      <p>Welcome to the MERN Stack application.</p>
    </div>
  );
}

export default Home;
```

In About.jsx

```
function About() {
  return (
    <div>
      <h1>About Page</h1>
      <p>This application demonstrates routing using React Router.</p>
    </div>
  );
}

export default About;
```

In Contact.jsx

```
function Contact() {
  return (
```

```
    <div>
      <h1>Contact Page</h1>
      <p>Email: mern@example.com</p>
    </div>
  );
}

export default Contact;
```

5. Configure BrowserRouter

In main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

In App.jsx

```
import { Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

function App() {
  return (
    <div>
      <h1>React Router Application</h1>

      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/about">About</Link> |{" "}
        <Link to="/contact">Contact</Link>
```

```
      </nav>

      <hr />

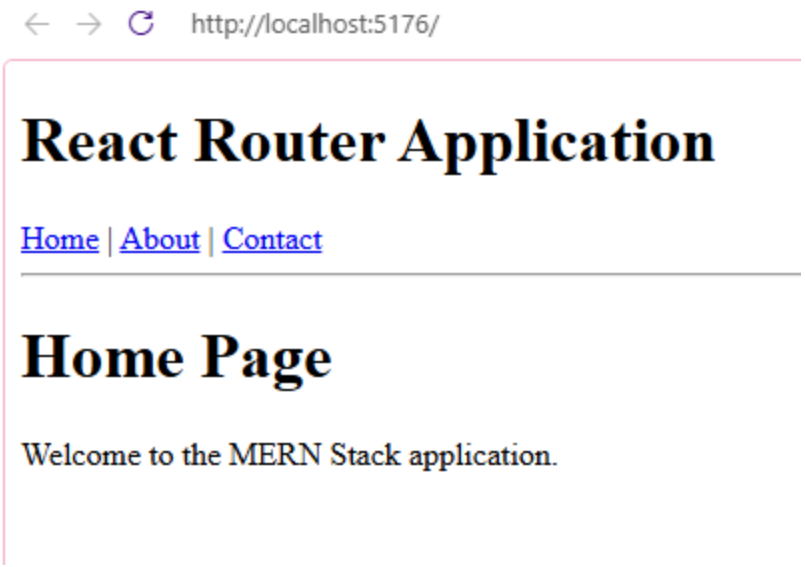
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </div>
  );
}

export default App;
```

6. Run the Application

If the development server is not running, execute:
npm run dev
Open the URL displayed in the terminal:
http://localhost:5176/

Output



b. Create multiple pages and implement navigation between them.

1. Create page Components

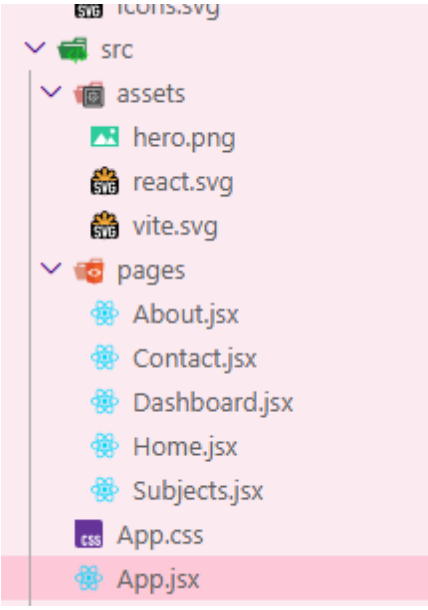
Inside the Src folder:

Pages

Inside it, create:

Dashboard.jsx

Subject.jsx



Dashboard.jsx

```
function Dashboard() {
  return (
    <div>
      <h2>Student Dashboard</h2>
      <p>Name: Rahul Sharma</p>
      <p>Course: B.Sc. Information Technology</p>
      <p>Semester: V</p>
    </div>
  );
}

export default Dashboard;
```

Subject.jsx

```jsx
function Subjects() {
  return (
    <div>
      <h2>Subjects</h2>
      <ul>
        <li>Full Stack Development</li>
        <li>Artificial Intelligence</li>
        <li>Information Security</li>
      </ul>
    </div>
  );
}

export default Subjects;
```

App.jsx

```jsx
import { Routes, Route, Link } from "react-router-dom";
import Dashboard from "./pages/Dashboard";
import Subjects from "./pages/Subjects";

function App() {
  return (
    <div>
      <h1>Student Portal</h1>
      <nav>
        <Link to="/">Dashboard</Link> |{" "}
        <Link to="/subjects">Subjects</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/subjects" element={<Subjects />} />
      </Routes>
    </div>
  );
}

export default App;
```

Output

Student Portal

Dashboard | Subjects

Student Dashboard

Name: Rahul Sharma

Course: B.Sc. Information Technology

Semester: V

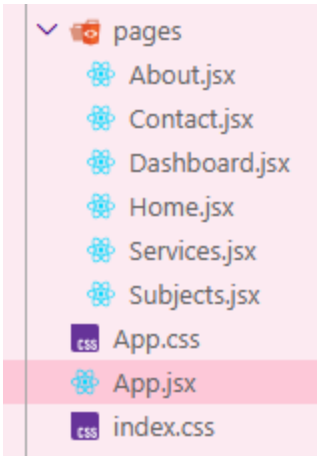
Student Portal

Dashboard | Subjects

Subjects

- Full Stack Development
- Artificial Intelligence
- Information Security

c. Use Link and Routes to move between pages.



Create Services.jsx

```
function Services() {
  return (
    <div>
      <h2>Our Services</h2>
      <ul>
        <li>Web Application Development</li>
        <li>Database Management</li>
        <li>API Development</li>
      </ul>
    </div>
  );
}

export default Services;
```

Contact.jsx

```
function Contact() {
  return (
    <div>
      <h2>Contact Us</h2>
      <p>Email: support@example.com</p>
      <p>Phone: 9876543210</p>
    </div>
  );
}
```

```
export default Contact;
```

App.jsx

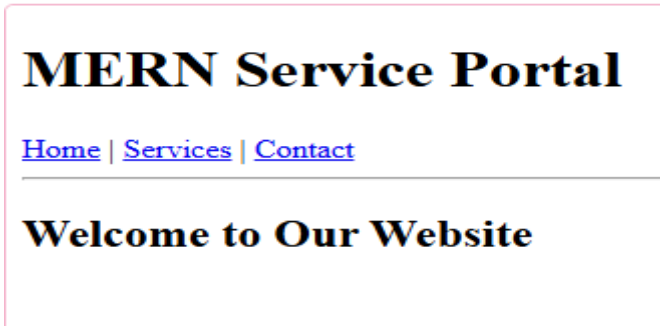
```
import { Routes, Route, Link } from "react-router-dom";
import Services from "./pages/Services";
import Contact from "./pages/Contact";

function App() {
  return (
    <div>
      <h1>MERN Service Portal</h1>
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/services">Services</Link> |{" "}
        <Link to="/contact">Contact</Link>
      </nav>
      <hr />

      <Routes>
        <Route path="/" element={<h2>Welcome to Our Website</h2>} />
        <Route path="/services" element={<Services />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </div>
  );
}

export default App;
```

Output



MERN Service Portal

Home | Services | Contact

Our Services

- Web Application Development
- Database Management
- API Development

MERN Service Portal

Home | Services | Contact

Contact Us

Email: support@example.com

Phone: 9876543210

d. Fetch sample data from a public API using Axios.

1. Install Axios

Execute:

npm install axios

```
C:\Users\Raj Salont\Desktop\MERN Practicals\practical-5>npm install axios

added 27 packages, and audited 59 packages in 1m

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

2. Replace App.jsx

```jsx
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.error("Error fetching data:", error);
      });
  }, []);

  return (
    <div>
      <h1>User Data</h1>
      {users.map((user) => (
        <div key={user.id}>
          <h3>{user.name}</h3>
          <p>Email: {user.email}</p>
        </div>
```

```
      ))}
    </div>
  );
}

export default App;
```

Output

User Data

Leanne Graham

Email: Sincere@april.biz

Ervin Howell

Email: Shanna@melissa.tv

Clementine Bauch

Email: Nathan@yesenia.net

Patricia Lebsack

Email: Julianne.OConner@kory.org

Chelsey Dietrich

Email: Lucio_Hettinger@annie.ca

Mrs. Dennis Schulist

Email: Karley_Dach@jasper.info

Kurtis Weissnat

a. Display fetched data in table or list format.

1. Replace App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.error("Unable to retrieve users:", error);
      });
  }, []);

  return (
    <div className="container">
      <h1>User Information</h1>
      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {users.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.username}</td>
              <td>{user.email}</td>
```

```
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

App.css

```
.container {
  width: 90%;
  margin: 30px auto;
  text-align: center;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  border: 1px solid black;
  padding: 10px;
  text-align: left;
}

th {
  background-color: lightgray;
}
```

Output

User Information

ID	Name	Username	Email	City
1	Leanne Graham	Bret	Sincere@april.biz	Gwenborough
2	Ervin Howell	Antonette	Shanna@melissa.tv	Wisokyburgh
3	Clementine Bauch	Samantha	Nathan@yesenia.net	McKenziehaven
4	Patricia Lebsack	Karianne	Julianne.OConner@kory.org	South Elvis
5	Chelsey Dietrich	Kamren	Lucio_Hettinger@annie.ca	Roscoeview
6	Mrs. Dennis Schulist	Leopoldo_Corkery	Karley_Dach@jasper.info	South Christy
7	Kurtis Weissnat	Elwyn.Skiles	Telly.Hoeger@billy.biz	Howemouth
8	Nicholas Runolfsdottir V	Maxime_Nienow	Sherwood@rosamond.me	Aliyaview
9	Glenna Reichert	Delphine	Chaim_McDermott@dana.io	Bartholomebury
10	Clementina DuBuque	Moriah.Stanton	Rey.Padberg@karina.biz	Lebsackbury

b. Implement search or filter functionality on displayed data.

1. Replace App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [users, setUsers] = useState([]);
  const [searchText, setSearchText] = useState("");

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.error("Unable to retrieve users:", error);
      });
  }, []);

  const filteredUsers = users.filter((user) =>
    user.name.toLowerCase().includes(searchText.toLowerCase())
  );

  return (
    <div className="container">
      <h1>Search User</h1>

      <input
        type="text"
        placeholder="Enter user name"
        value={searchText}
        onChange={(event) => setSearchText(event.target.value)}
      />

      <table>
        <thead>
          <tr>
            <th>Name</th>
```

```
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

App.css

```
.container {
  width: 85%;
  margin: 30px auto;
  text-align: center;
}

input {
  width: 300px;
  padding: 10px;
  margin-bottom: 20px;
  font-size: 16px;
}

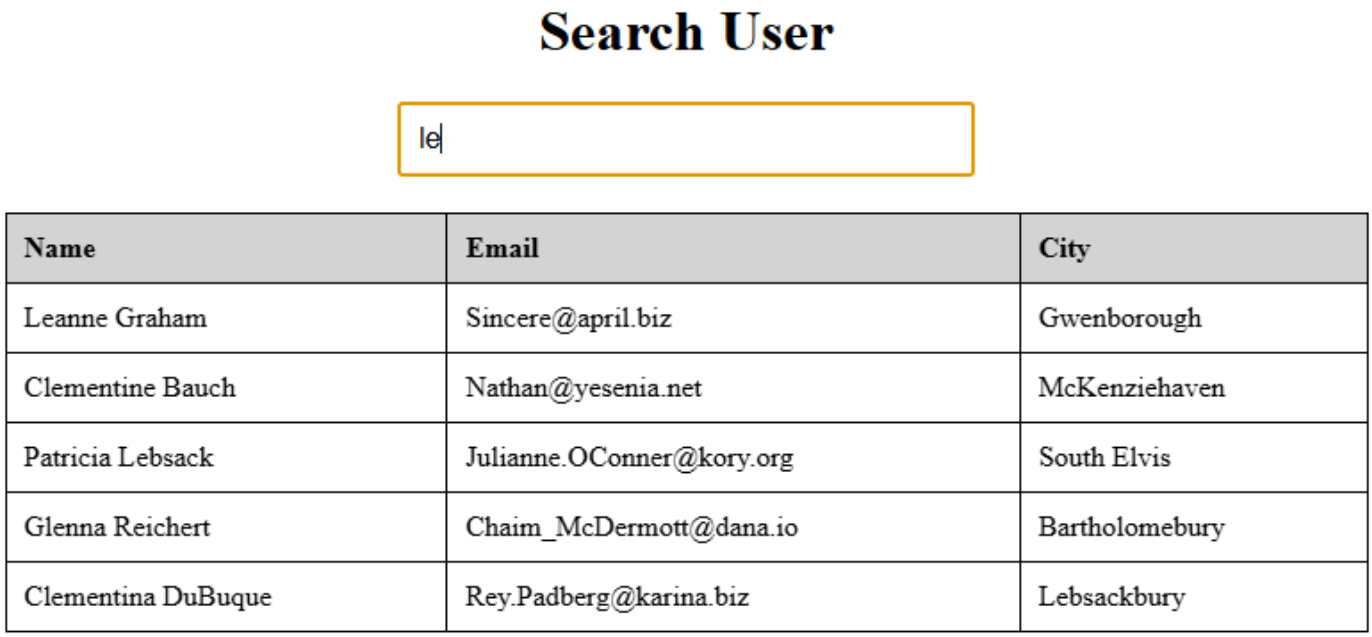
table {
  width: 100%;
  border-collapse: collapse;
}

th,
```

```css
td {
  border: 1px solid black;
  padding: 10px;
  text-align: left;
}

th {
  background-color: lightgray;
}
```

Output



Important Methods / Functions

1. `BrowserRouter`

Used to wrap the application and enable routing functionality for Single Page Application.

2. `Routes`

Used as a container to hold all the individual Route definitions.

3. `Route`

Used to define a single page path and which component to show on that path.

4. `Link to="/path"`

Used to navigate between pages without reloading the browser page.

5. `useEffect()`

Used to perform side effects like fetching data from an API when component loads.

6. `axios.get()`

Used to fetch sample data from a public API using Axios library.

7. `useState()`

Used to store the fetched API data in state.

8. `filter()`

Used to implement search or filter functionality on the displayed data.

9. `then()`

Used to handle the response received from the API after fetching.

Module 2: Backend Development Using Node.js, Express, and MongoDB

1. Node.js and Express Fundamentals

a. Install Express and create a simple server.

Procedure

Since this is the first Express practical, the initial project setup and Express installation will be explained step by step.

1 : Create the Practical Folder

Inside the MERN_Practicals folder, create a new folder named: Practical-1

(ignore Practical-6 in the below snapshot)

Open this folder in Visual Studio Code.

2 : Open the Integrated Terminal

Select: Terminal -> New Terminal

The terminal should display a path similar to:

PS C:\MERN_Practicals\Practical_1>

```
C:\Users\Raj Salont\Desktop\MERN Backend>cd practical-6

C:\Users\Raj Salont\Desktop\MERN Backend\practical-6>
```

3 : Initialize the Node.js Project

Execute the init command in the terminal.

npm init -y

This command creates a package.json file containing the basic information and dependencies of the Node.js project.

```
C:\Users\Raj Salont\Desktop\MERN Backend\practical-6>npm init -y
Wrote to C:\Users\Raj Salont\Desktop\MERN Backend\practical-6\package.json:

{
  "name": "practical-6",
  "version": "1.0.0",
  "description": "",
  "license": "ISC",
  "author": "",
  "type": "commonjs",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": []
}



C:\Users\Raj Salont\Desktop\MERN Backend\practical-6>npm install express

added 68 packages, and audited 69 packages in 17s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Raj Salont\Desktop\MERN Backend\practical-6>
```

4 : Install Express.js

Execute

the install command for express.(npm install express)

After successful installation, Express will be added to the project dependencies.

5 : Create the Server File

Create a new file named: server.js

The project structure will appear similar to:

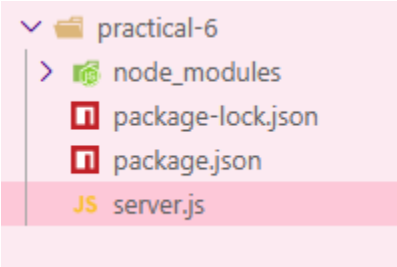
Practical_1 containing

node_modules

package-lock.json

package.json

server.js



6 : Write the Express Server Program

Open server.js and enter the server program.

```
const express = require("express");

const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
  res.send("Welcome to MERN Stack Backend Development");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

7 : Run the Server

Save the file and execute the file using node.

The terminal displays:

Server is running on port 3000

8 : Open the Application in the Browser

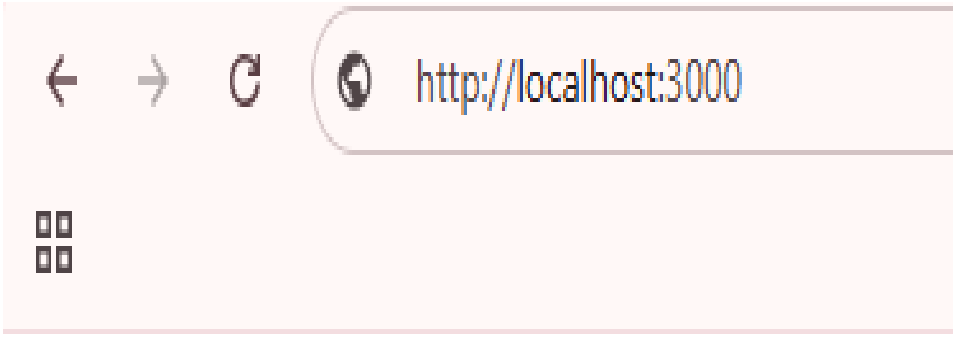
Open the following address in the browser:

http://localhost:3000

The browser displays the response sent by the Express server.

```
C:\Program Files\nodejs\node.exe .\practical-6\server.js
Server running on port 3000
```

Output



Welcome to MERN Stack Backend Development

b. Create Multiple Routes in Express

Procedure:

Create multiple routes like /, /about, /courses, /contact

in server.js

```
const express = require("express");

const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
  res.send("Welcome to the Home Page");
});

app.get("/about", (req, res) => {
  res.send("About MERN Stack Development");
});

app.get("/courses", (req, res) => {
  res.send("Course: B.Sc. Information Technology");
});

app.get("/contact", (req, res) => {
  res.send("Contact: mern@example.com");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

run the server.

Open the URLs individually in browser to see output.

output.

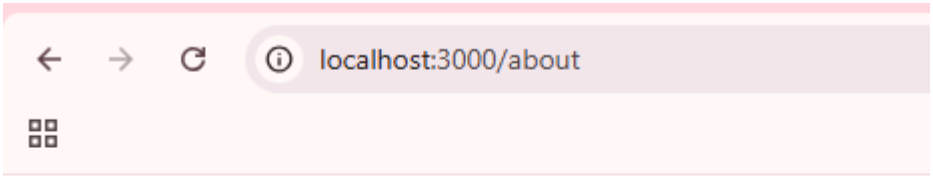


Welcome to MERN Stack Backend Development

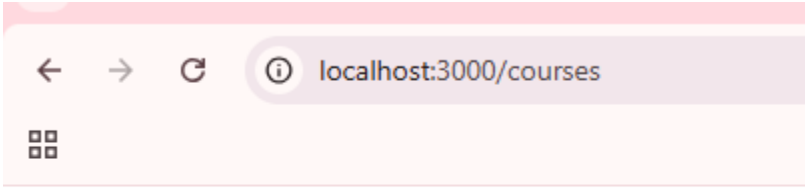
Error alert: you have to restart the server in Express every time you change the code

In terminal

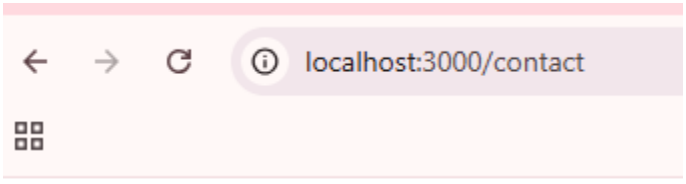
1. ctrl + C (closes the server) and save the file
2. then bash node server.js



About MERN Stack Development



Course: B.Sc. Information Technology



Contact: mern@example.com

C. Send HTML Content from Express Server

Procedure:

Create a route that sends HTML content

in server.js

```
const express = require("express");

const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
  res.send(`
    <h1>MERN Stack Development</h1>
    <h2>Backend Using Express.js</h2>
    <p>Welcome to the Express server.</p>
    <p>Course: B.Sc. Information Technology</p>
    <p>Semester: V</p>
  `);
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

open local host:3000

Output

MERN Stack Development

Backend Using Express.js

Welcome to the Express server.

Course: B.Sc. Information Technology

Semester: V

d. Handle Route Parameters in Express

in server.js

```
const express = require("express");

const app = express();
const PORT = 3000;

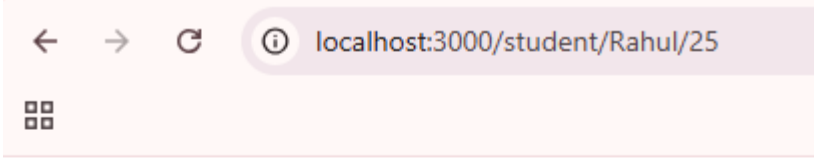
app.get("/student/:name/:rollno", (req, res) => {
  const studentName = req.params.name;
  const rollNo = req.params.rollno;

  res.send(`
    <h1>Student Information</h1>
    <p>Name: ${studentName}</p>
    <p>Roll Number: ${rollNo}</p>
  `);
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

http://localhost:3000/student/Rahul/25

Output



Student Information

Name: Rahul

Roll Number: 25

e. Handle Query Parameters in Express

in server.js

```
const express = require("express");

const app = express();
const PORT = 3000;

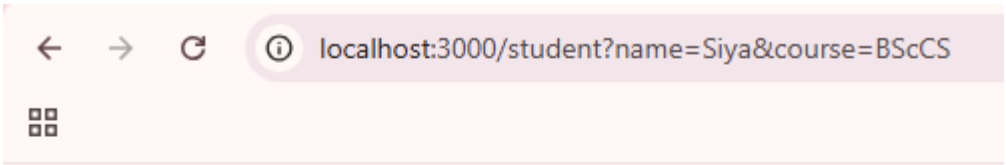
app.get("/student", (req, res) => {
  const name = req.query.name;
  const course = req.query.course;

  res.send(`
    <h1>Student Details</h1>
    <p>Name: ${name}</p>
    <p>Course: ${course}</p>
  `);
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

http://localhost:3000/student?name=Siya&course=BScCS

Output



Student Details

Name: Siya

Course: BScCS

f. Create a Simple JSON API using Express

in server.js

```
const express = require("express");

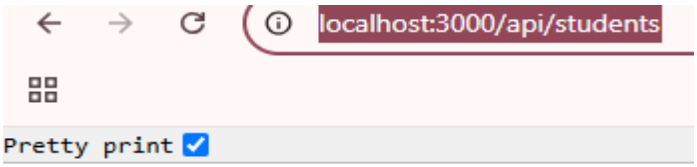
const app = express();
const PORT = 3000;

const students = [
  { id: 1, name: "Amit", course: "B.Sc. IT", semester: 5 },
  { id: 2, name: "Priya", course: "B.Sc. IT", semester: 5 },
  { id: 3, name: "Neha", course: "B.Sc. IT", semester: 5 }
];

app.get("/api/students", (req, res) => {
  res.json(students);
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

http://localhost:3000/api/students




```
[
  {
    "id": 1,
    "name": "Amit",
    "course": "B.Sc. IT",
    "semester": 5
  },
  {
    "id": 2,
    "name": "Priya",
    "course": "B.Sc. IT",
    "semester": 5
  },
  {
    "id": 3,
    "name": "Neha",
    "course": "B.Sc. IT",
    "semester": 5
  }
]
```

2. MongoDB Installation and Database Operations

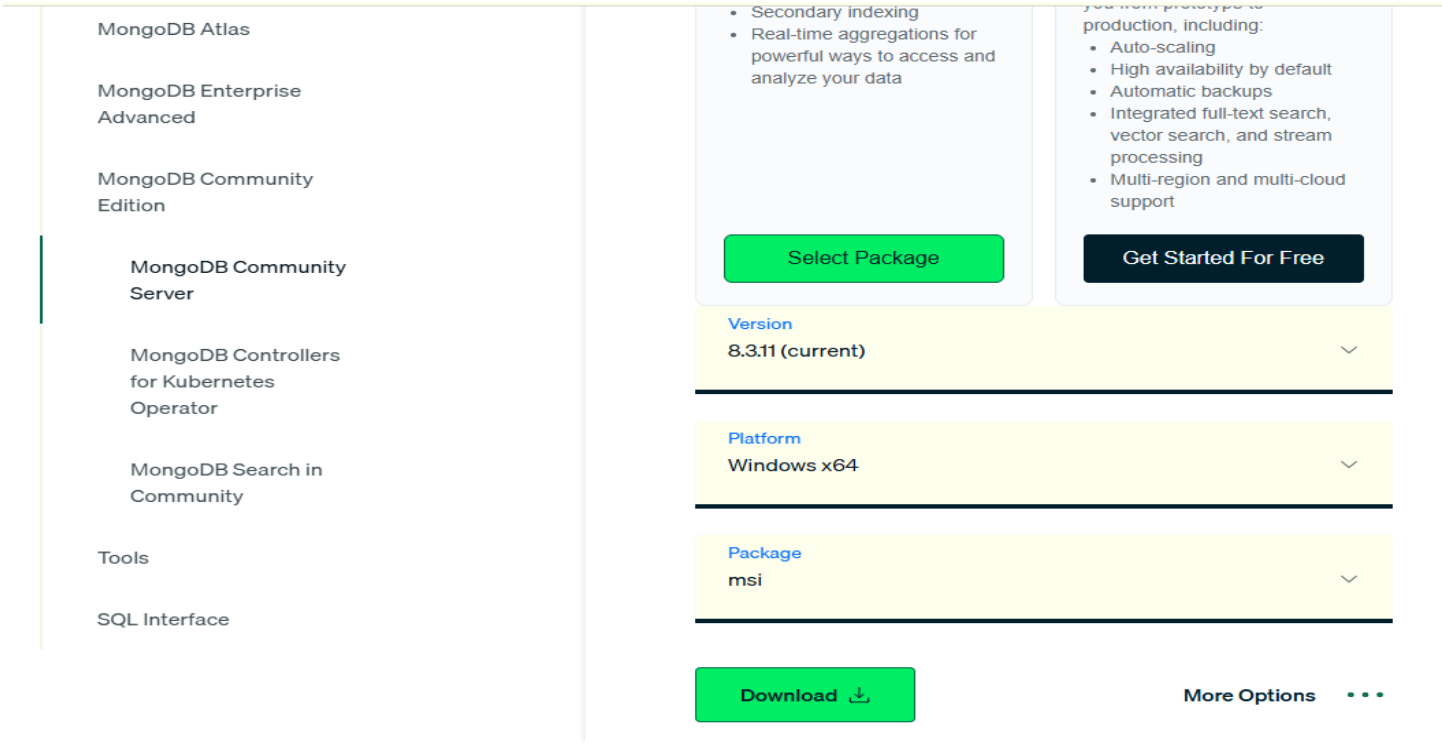
a. Install MongoDB Community Server and MongoDB Compass.

MongoDB is a NoSQL, document-oriented database used in MERN Stack.

Unlike SQL databases where data is stored in tables and rows, MongoDB stores data in collections and documents.

1. Download MongoDB Community Server

https://www.mongodb.com/try/download/community



Choose Complete installation

Install MongoDB Compass

Finish

Open MongoDB Compass

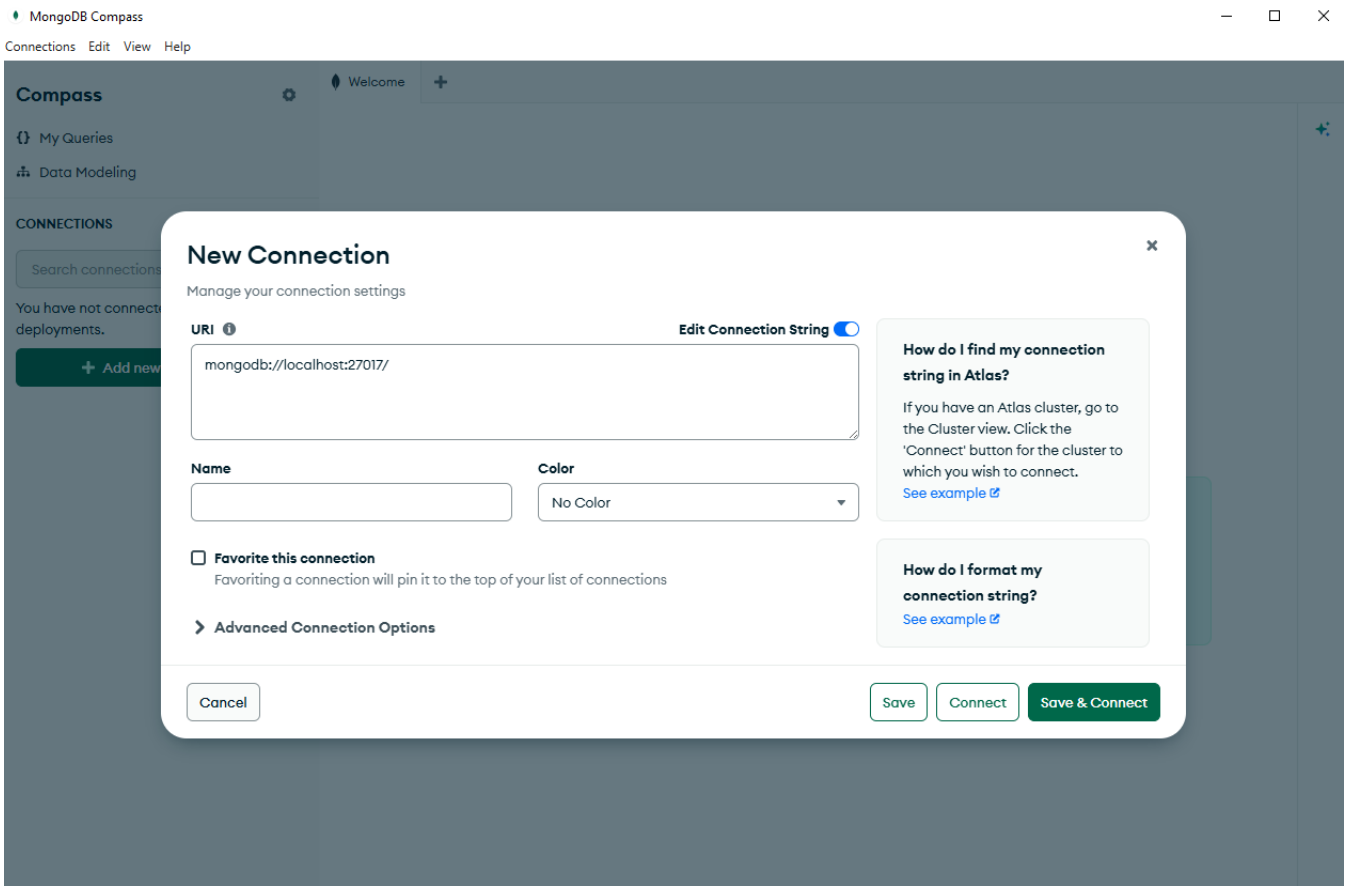
Launch MongoDB Compass.

The default connection string appears as:

mongodb://localhost:27017

Click

Connect



Create a Database

Click Create Database

Enter:

Database Name

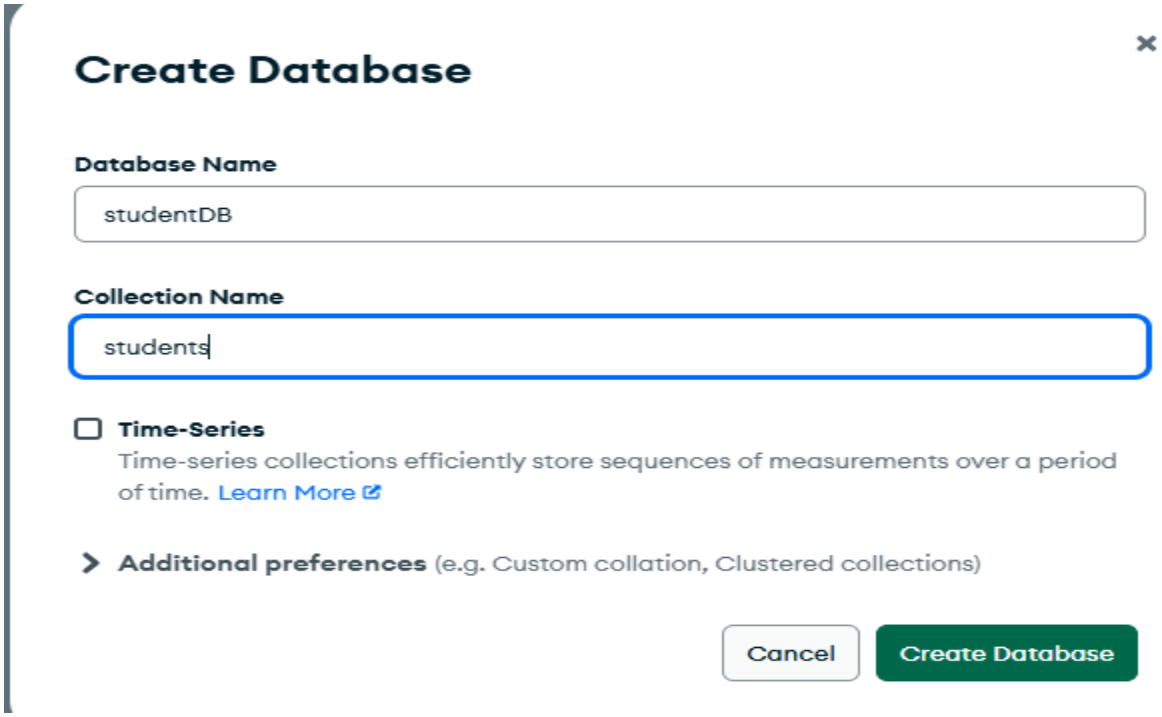
studentDB

Collection Name

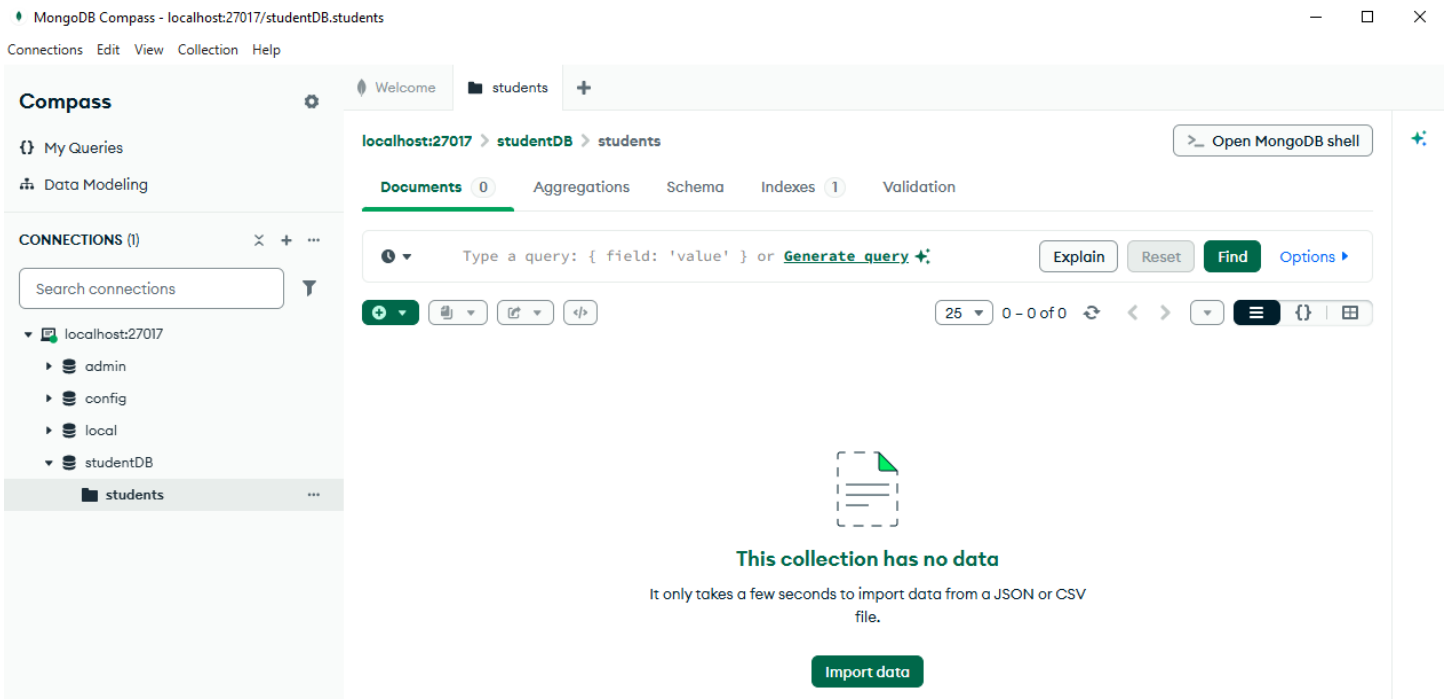
students

Click

Create Database



Creating the studentDB Database



b. Perform CRUD operations using MongoDB Compass.

Program / Operations

Insert the following document into the students collection.

{

"studentId": 101,

"name": "Rahul Sharma",

"course": "B.Sc. Information Technology",

"semester": 5,

"city": "Mumbai"

}

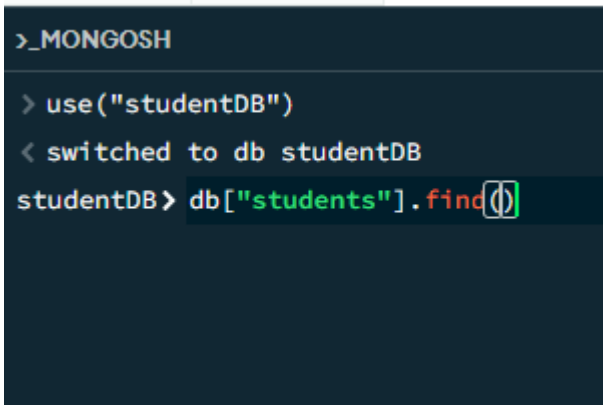
Perform the following operations:

Insert a document

View documents

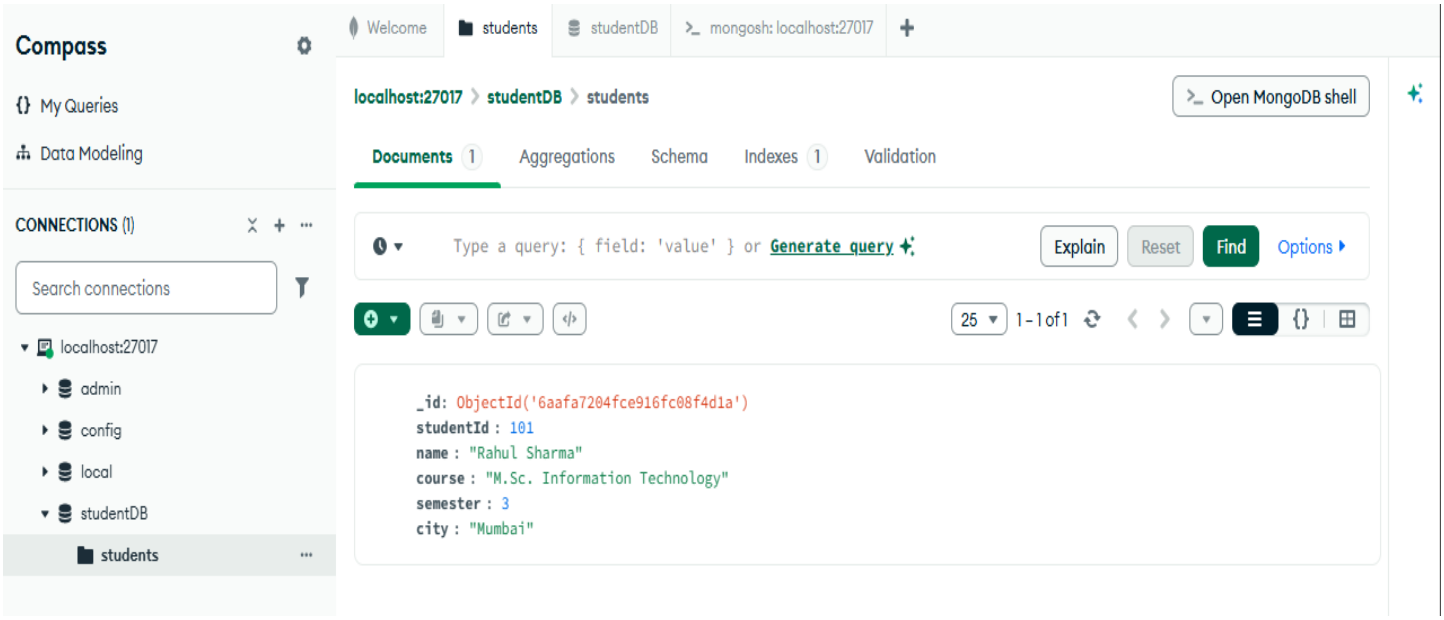
Update the city field

Delete the document



```
>_MONGOSH
> use("studentDB")
< switched to db studentDB
> db["students"].find()
<
>
> db.students.insertOne({ studentId: 101, name: "Rahul Sharma", course: "M.Sc. Information Technology", semester: 3,
< {
    acknowledged: true,
    insertedId: ObjectId('6aafa7204fce916fc08f4d1a')
  }
> db.students.find()
< {
    _id: ObjectId('6aafa7204fce916fc08f4d1a'),
    studentId: 101,
    name: 'Rahul Sharma',
    course: 'M.Sc. Information Technology',
    semester: 3,
    city: 'Mumbai'
  }
studentDB>
```

Output:



c. Connect Node.js with MongoDB using Mongoose.

Create a folder named MONGO

Open the integrated terminal

Bash

Cd MONGO

npm init –y

npm install mongoose

```
C:\Users\Raj Salont\Desktop\MERN Backend>cd MONGO

C:\Users\Raj Salont\Desktop\MERN Backend\MONGO>npm init -y
Wrote to C:\Users\Raj Salont\Desktop\MERN Backend\MONGO\package.json:

{
  "name": "mongo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}



C:\Users\Raj Salont\Desktop\MERN Backend\MONGO>npm install mongoose

added 19 packages, and audited 20 packages in 11s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Raj Salont\Desktop\MERN Backend\MONGO>
```

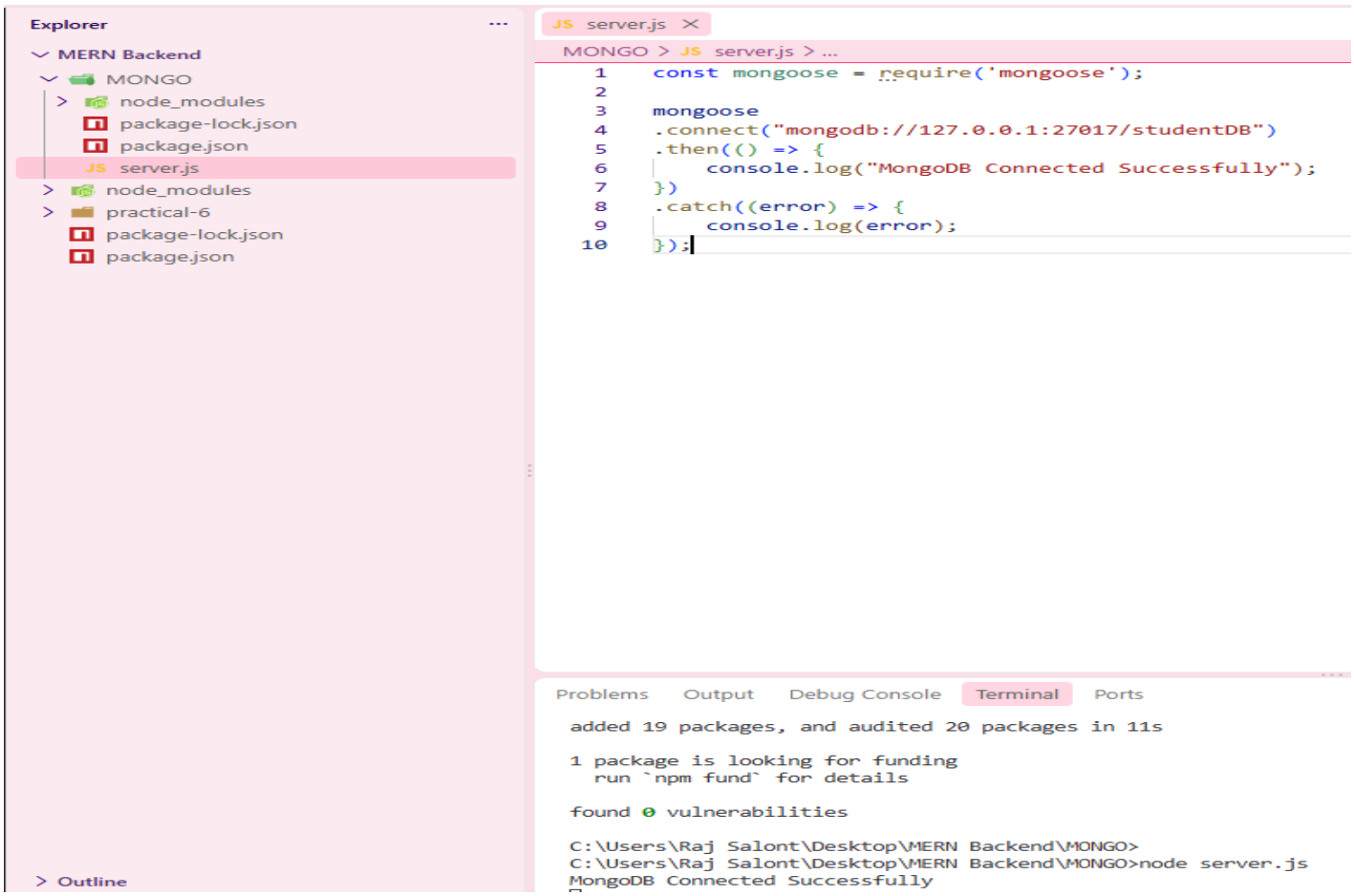
Create a file inside MONGO folder named server.js

```
const mongoose = require('mongoose');
```

```
mongoose
.connect("mongodb://127.0.0.1:27017/studentDB")
.then(() => {
    console.log("MongoDB Connected Successfully");
})
.catch((error) => {
    console.log(error);
});
```

Run:

Node server.js



3. API Creation and Integration

a. Create a MongoDB Scheme and model using Mongoose.

Procedure

1. Open the Practical-3 folder in Visual Studio Code.

2 Open the integrated terminal and initialize the project.

npm init –y

3 Install the required packages.

npm install express mongoose

```
C:\Users\Raj Salont\Desktop\MERN Backend\practical-3>npm install express mongoose

added 86 packages, and audited 87 packages in 25s

29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

4. Create a file named

Server.js

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();

app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/studentDB")
.then(()=> console.log("MongoDB Connected Successfully"))
.catch(err => console.log(err));

const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  semester: Number,
  city: String
```

```
});

const Student = mongoose.model("Student", studentSchema);

app.listen(3000, () => {
  console.log("Server Running on Port 3000");
});
```

Node.server.js

```
C:\Users\Raj Salont\Desktop\MERN Backend\practical-3>node server.js
Server Running on Port 3000
MongoDB Connected Successfully
```

b. Implement REST API Operations.

In server.js

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();

app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/studentDB")
.then(()=> console.log("MongoDB Connected Successfully"))
.catch(err => console.log(err));

const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  semester: Number,
  city: String
});

const Student = mongoose.model("Student", studentSchema);

// POST
app.post("/students", async (req, res) => {
  const student = new Student(req.body);
  await student.save();
  res.json(student);
});

// GET
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// PUT
app.put("/students/:id", async (req, res) => {
  const student = await Student.findByIdAndUpdate(
    req.params.id,
    req.body,
```

```
    { new: true }
  );
  res.json(student);
});

// DELETE
app.delete("/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);
  res.json({
    message: "Student Deleted Successfully"
  });
});

app.listen(3000, () => {
  console.log("Server Running on Port 3000");
});
```

Output

Execute:

Node.server.js

```
C:\Users\Raj Salont\Desktop\MERN Backend>cd practical-3

C:\Users\Raj Salont\Desktop\MERN Backend\practical-3>node server.js
Server Running on Port 3000
MongoDB Connected Successfully
```

c. Test REST APIs Using Postman.

Procedure

Install Postman

1 Open your web browser and visit:

postman.com

https://www.postman.com/downloads/

2 Download the Windows 64-bit version.

3 Run the downloaded installer.

4 Follow the installation wizard and click Install.

5 After installation open Postman.

Skip Login (Recommended for Students)

When Postman opens, it may ask you to sign in.

Choose one of these options:

Skip and go to the app

Continue without an account

(Depending on the version, the wording may differ.)

Create Your First Request

Click New or the + tab.

Select HTTP Request.

At the top, change the request method to **GET**.

Enter the URL:

localhost:3000

Click Send.

If your Express server is running, you will see the response from your API.

Test POST Request

Change the method to POST.

Enter the URL:

localhost:3000/students

Click Body.

Select raw.

From the dropdown, choose JSON.

Enter:

```
{
"name": "Rahul Sharma",
"course": "B.Sc. Information Technology",
"semester": 5,
"city": "Mumbai"
}
```

Click Send.

If successful, the inserted document will be displayed in the response.

Open Postman and test the following requests.

Method	**URL**
GET	http://localhost:3000/students
PUT	http://localhost:3000/students/:id
DELETE	http://localhost:3000/students/:id

Output

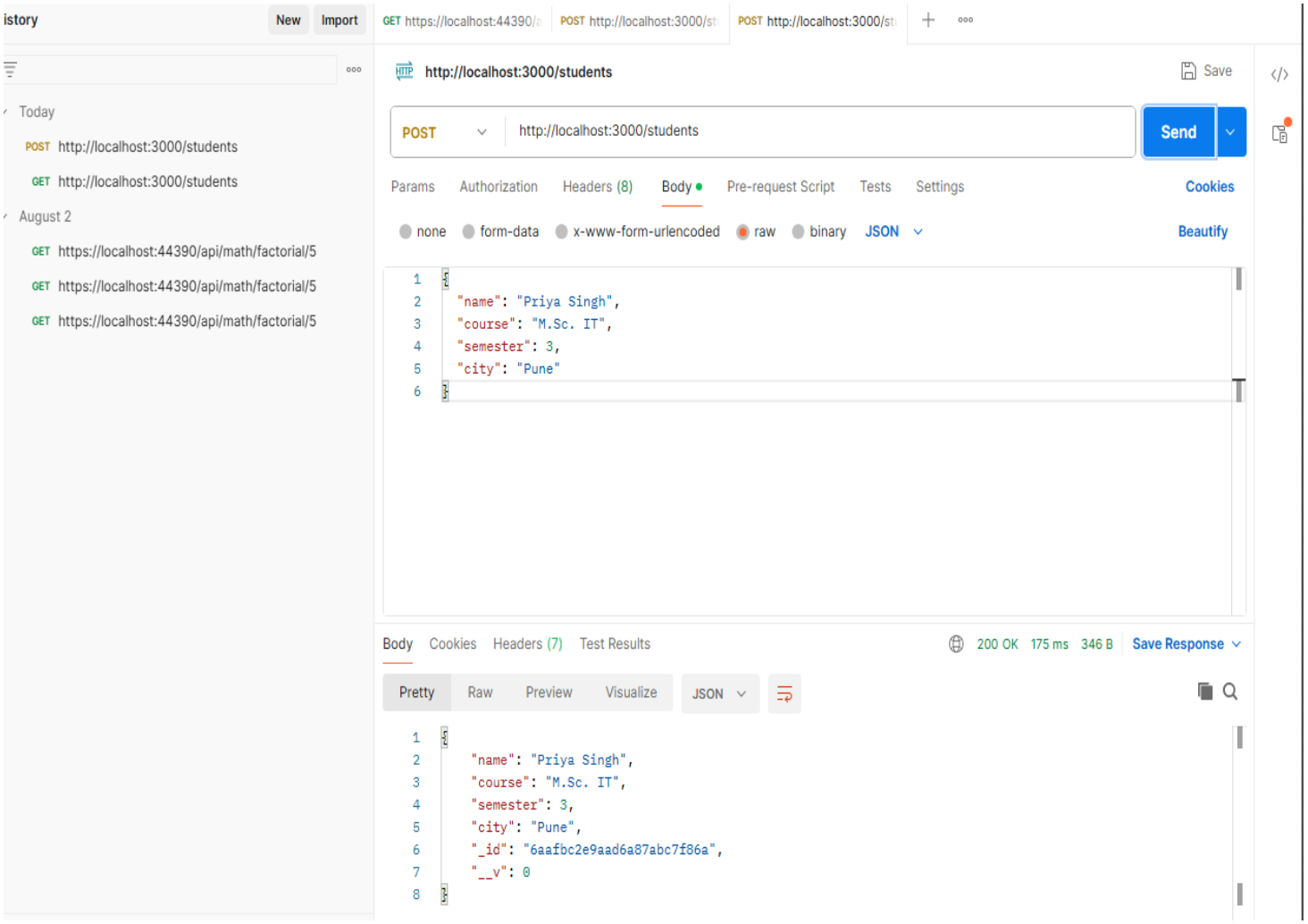
Perform the following operations in Postman:

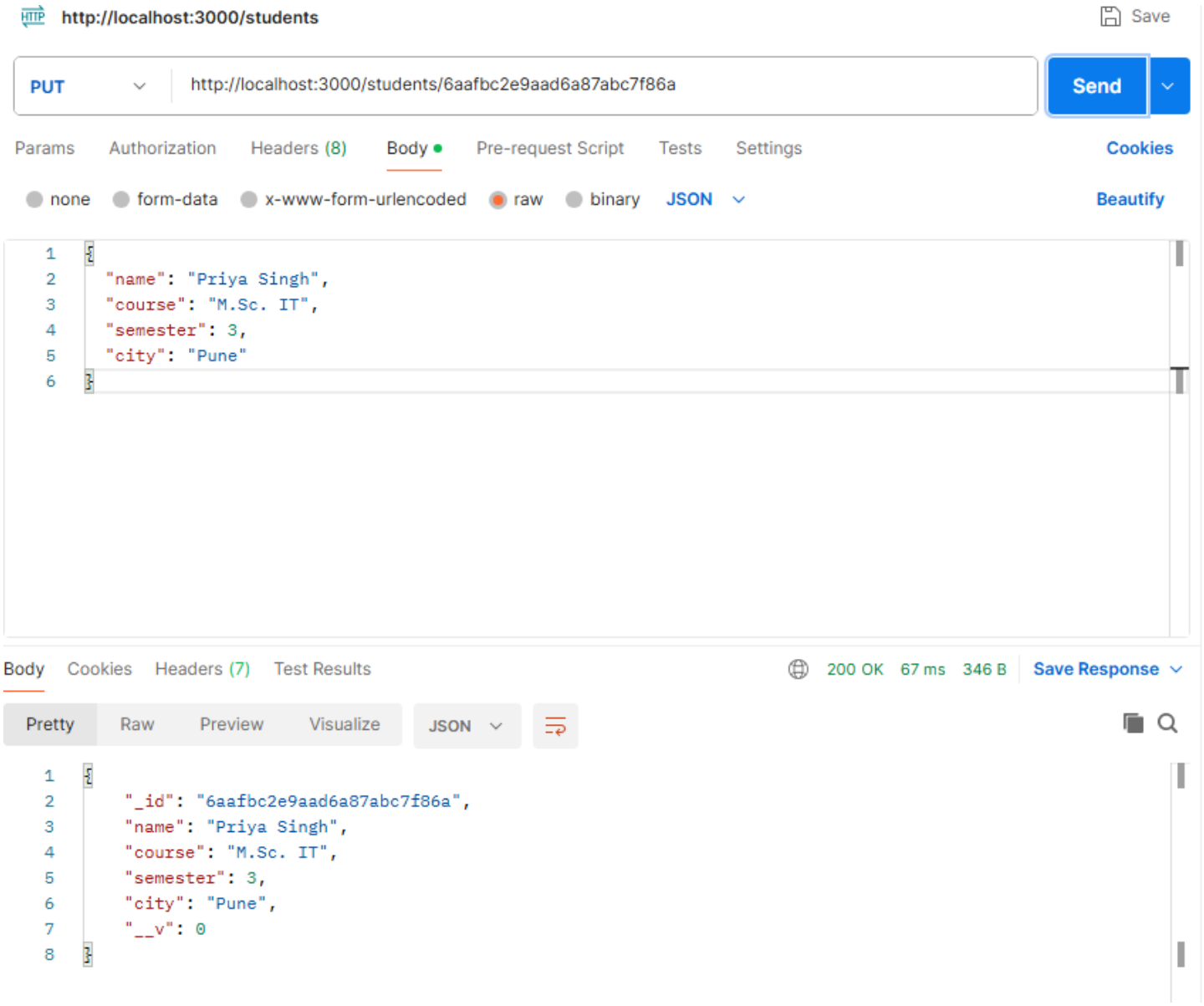
Insert a student record.

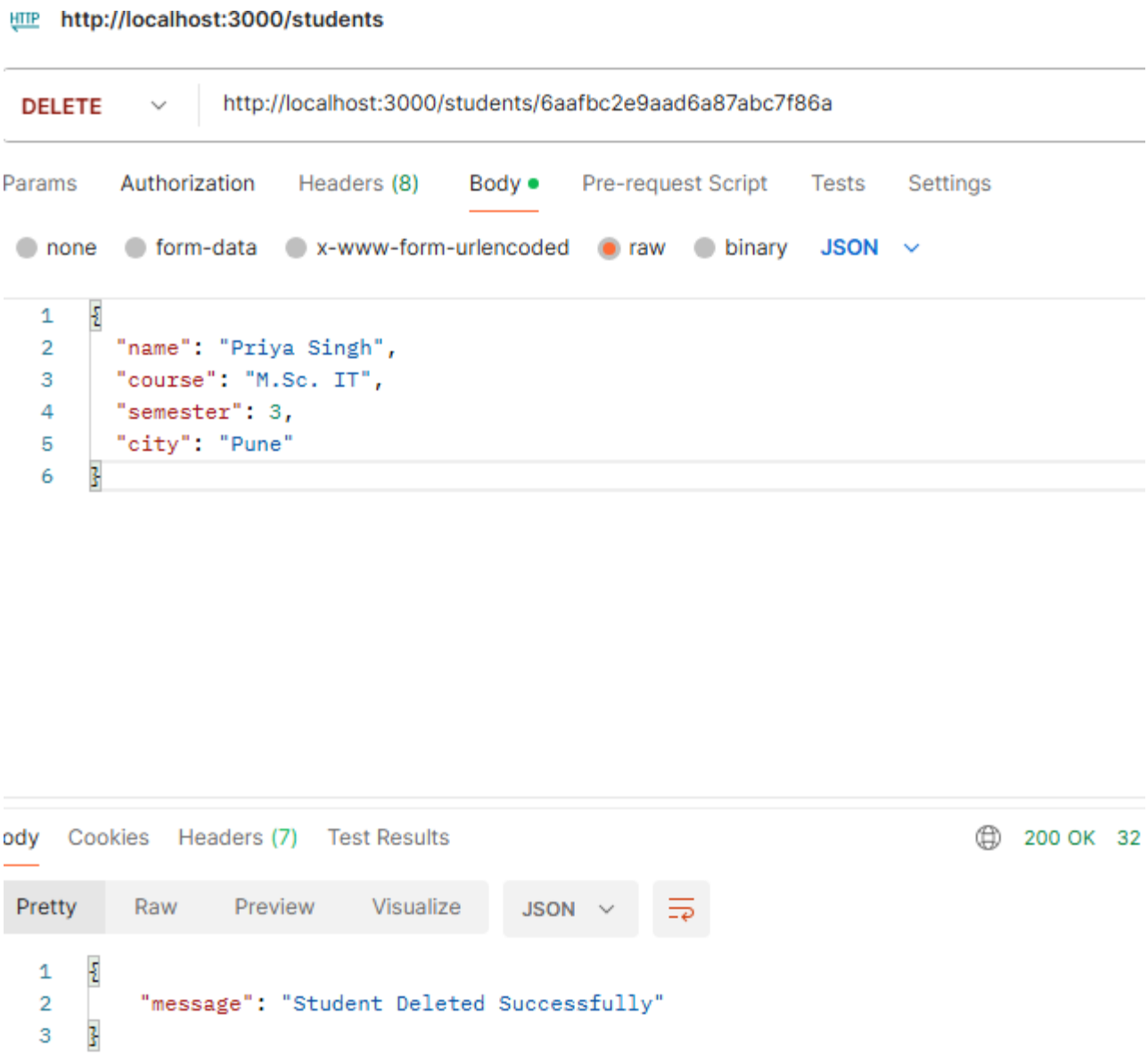
Retrieve all student records.

Update a student record.

Delete a student record.


istory
New
Import
Today
POST http://localhost:3000/students
GET http://localhost:3000/students
August 2
GET https://localhost:44390/api/math/factorial/5
GET https://localhost:44390/api/math/factorial/5
GET https://localhost:44390/api/math/factorial/5
http://localhost:3000/students
Save
POST
http://localhost:3000/students
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"name": "Priya Singh",
"course": "M.Sc. IT",
"semester": 3,
"city": "Pune"
}
Body
Cookies
Headers (7)
Test Results
200 OK
175 ms
346 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"name": "Priya Singh",
"course": "M.Sc. IT",
"semester": 3,
"city": "Pune",
"_id": "6aafbc2e9aad6a87abc7f86a",
"__v": 0
}


http://localhost:3000/students
Save
PUT
http://localhost:3000/students/6aafbc2e9aad6a87abc7f86a
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"name": "Priya Singh",
"course": "M.Sc. IT",
"semester": 3,
"city": "Pune"
}
Body
Cookies
Headers (7)
Test Results
200 OK
67 ms
346 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"_id": "6aafbc2e9aad6a87abc7f86a",
"name": "Priya Singh",
"course": "M.Sc. IT",
"semester": 3,
"city": "Pune",
"__v": 0
}


http://localhost:3000/students
DELETE
http://localhost:3000/students/6aafbc2e9aad6a87abc7f86a
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
JSON
{
"name": "Priya Singh",
"course": "M.Sc. IT",
"semester": 3,
"city": "Pune"
}
ody
Cookies
Headers (7)
Test Results
200 OK
32
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Student Deleted Successfully"
}

4. Frontend and Backend Integration using React and Axios.

Axios is used to send HTTP requests from React to the Express API. The backend also requires CORS middleware so that the React application running on one port can communicate with the Express server running on another port. (npm)

a. Create the React Frontend and connect it with Express.

To create a React application and connect it to the Express backend using Axios.Procedure

1: Keep the Backend Server Ready

Open the existing Practical_3 folder in one VS Code window.

Make sure the backend server.js contains the CRUD API developed in Practical 3.

Install CORS in the backend folder:

npm install cors

Add this line near the top of server.js:

const cors = require("cors");

Add the following middleware after creating the Express application:

app.use(cors());

app.use(express.json());

The beginning of the backend file should appear as follows:

```
1  const express = require("express");
2  const mongoose = require("mongoose");
3  const cors = require("cors");
4
5  const app = express();
6
7  app.use(cors());
8  app.use(express.json());
9
10 mongoose.connect("mongodb://127.0.0.1:27017/studentDB")
11 .then(()=> console.log("MongoDB Connected Successfully"))
12 .catch(err => console.log(err));
```

Run the backend:

node server.js

The terminal should display:

```
found 0 vulnerabilities

C:\Users\Raj Salont\Desktop\MERN Backend\practical-8>node server.js
Server Running on Port 3000
MongoDB Connected Successfully
```

2: Create the React Application

Open another VS Code window.

Create a folder named:

Practical-4

Open the terminal and execute:

npm create vite@latest

Enter the project name:

student-frontend

Select:

Framework: React

Variant: JavaScript

Then execute:

cd student-frontend

npm install

Vite provides a quick way to create a React project using a React template. (vitejs)

3: Install Axios

Inside the student-frontend folder, run:

npm install axios

Axios is a promise-based HTTP client used to send requests such as GET, POST, PUT, and DELETE. (npm)

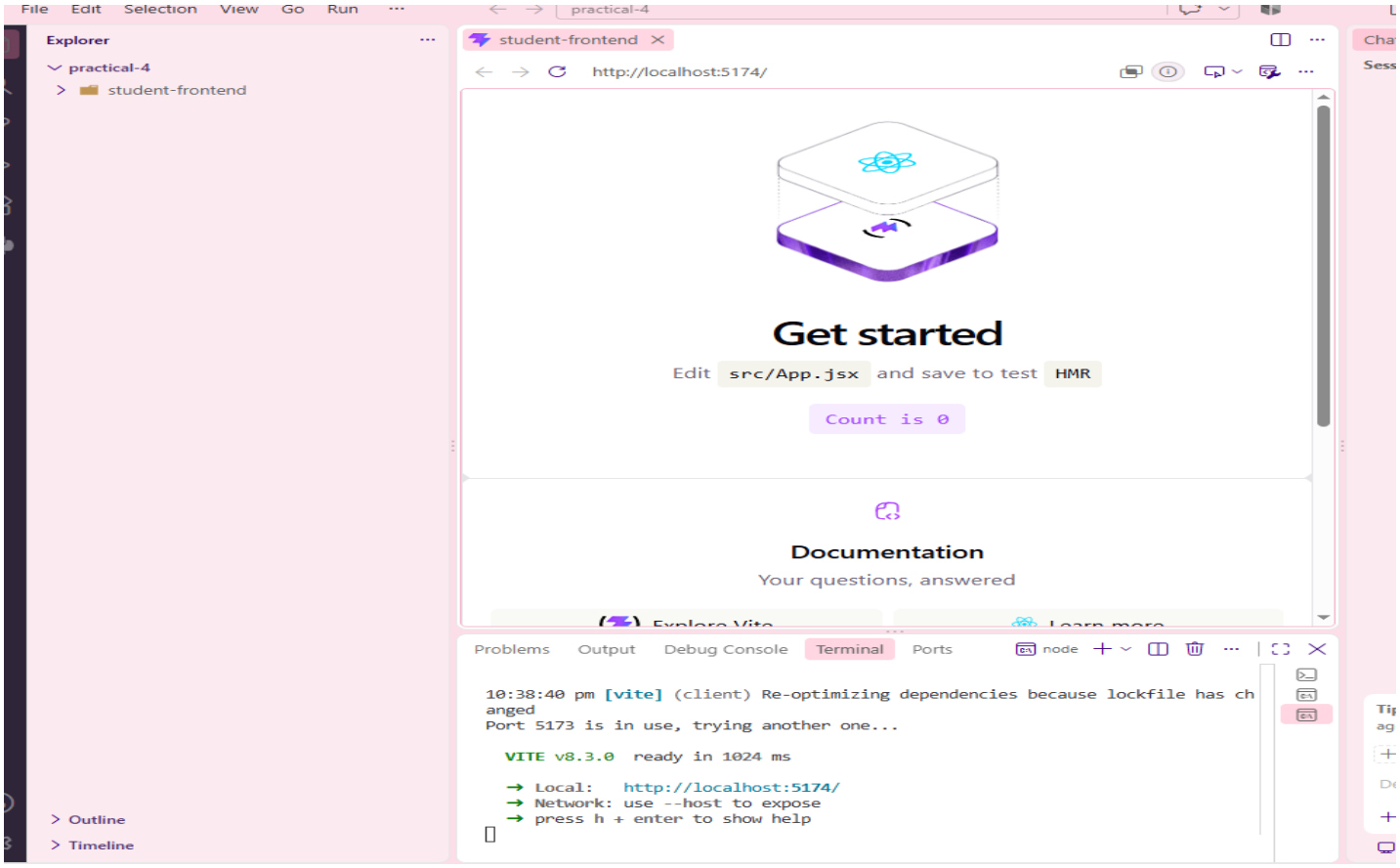
4: Start the React Application

Execute:

npm run dev

Open the displayed local URL, normally:

http://localhost:51734/



B. Retrieve and Display Student Records.

To retrieve student records from the Express backend and display them in a React application.

In App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [students, setStudents] = useState([]);

  const fetchStudents = async () => {
    try {
      const response = await axios.get(
        "http://localhost:3000/students"
      );
      setStudents(response.data);
    } catch (error) {
      console.error("Error fetching students:", error);
    }
  };

  useEffect(() => {
    fetchStudents();
  }, []);

  return (
    <div className="container">
      <h1>Student Management System</h1>

      <h2>Student Records Retrieved from Backend</h2>

      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Course</th>
            <th>Semester</th>
```

```
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student._id}>
              <td>{student.name}</td>
              <td>{student.course}</td>
              <td>{student.semester}</td>
              <td>{student.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

In App.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  margin: 0;
}

.container {
  width: 80%;
  margin: 40px auto;
  background-color: white;
  padding: 25px;
  border-radius: 8px;
}

h1 {
  text-align: center;
}

table {
```

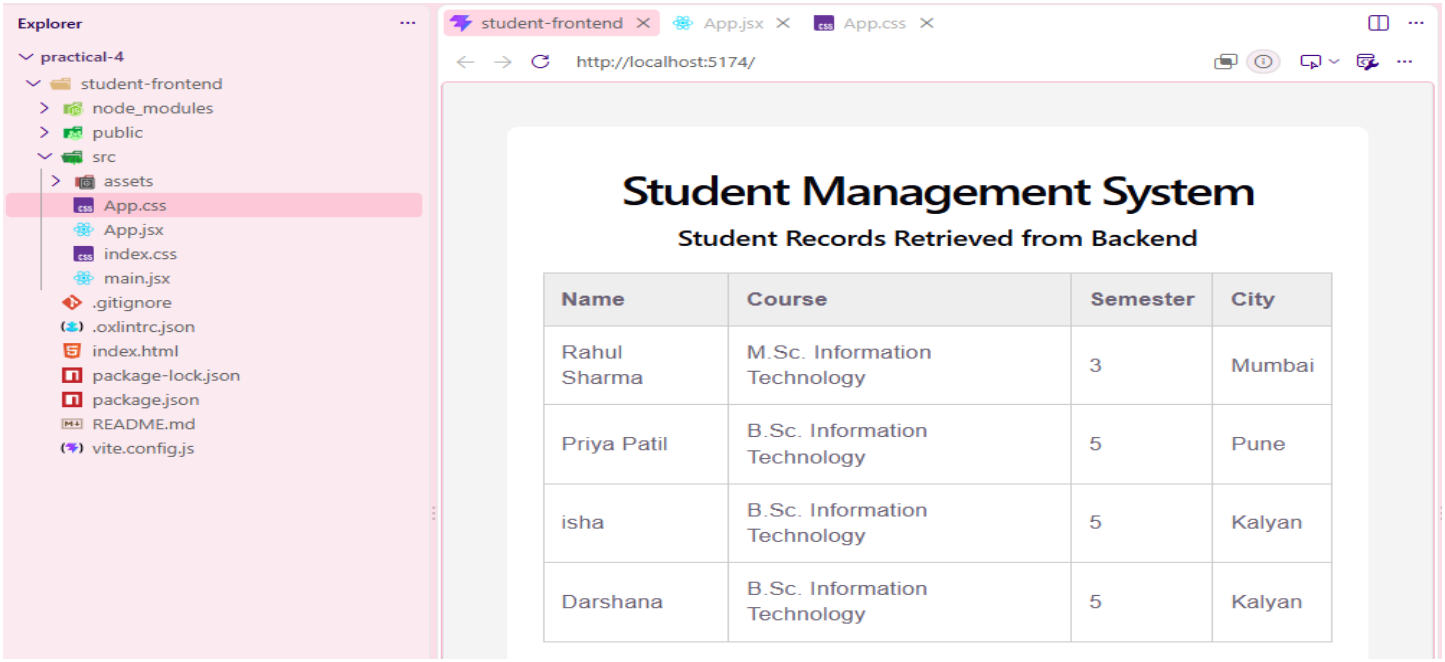
```
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
}

th,
td {
  border: 1px solid #cccccc;
  padding: 12px;
  text-align: left;
}

th {
  background-color: #eeeeee;
}
```

Output

The React application retrieves the student records stored in MongoDb and displays them in a table.



c. Add, Update and Delete Student Records.

In src/app.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [students, setStudents] = useState([]);
  const [formData, setFormData] = useState({ name: "", course: "", semester: "",
city: "" });
  const [editingId, setEditingId] = useState(null);
  const [message, setMessage] = useState("");
  const apiUrl = "http://localhost:3000/students";

  const fetchStudents = async () => {
    try {
      const response = await axios.get(apiUrl);
      setStudents(response.data);
    } catch (error) {
      console.error("Error fetching students:", error);
    }
  };

  useEffect(() => { fetchStudents(); }, []);

  const handleChange = (event) => {
    setFormData({ ...formData, [event.target.name]: event.target.value });
  };

  const handleSubmit = async (event) => {
    event.preventDefault();
    try {
      if (editingId) {
        await axios.put(`${apiUrl}/${editingId}`, formData);
        setEditingId(null);
        setMessage("Student updated successfully!");
      } else {
        await axios.post(apiUrl, formData);
        setMessage("Student added successfully!");
      }
```

```
      setFormData({ name: "", course: "", semester: "", city: "" });
      fetchStudents();
      setTimeout(() => setMessage(""), 3000);
    } catch (error) {
      console.error("Error saving student:", error);
      setMessage("Error saving student!");
    }
  };

  const handleEdit = (student) => {
    setFormData({ name: student.name, course: student.course, semester:
student.semester, city: student.city });
    setEditingId(student._id);
  };

  const handleDelete = async (id) => {
    try {
      await axios.delete(`${apiUrl}/${id}`);
      fetchStudents();
      setMessage("Student deleted successfully!");
      setTimeout(() => setMessage(""), 3000);
    } catch (error) {
      console.error("Error deleting student:", error);
    }
  };

  return (
    <div className="container">
      <h1>Student Management System</h1>
      {message && <p style={{ color: 'green', textAlign: 'center', fontWeight:
'bold' }}>{message}</p>}

      <form onSubmit={handleSubmit}>
        <input type="text" name="name" placeholder="Enter student name"
value={formData.name} onChange={handleChange} required />
        <input type="text" name="course" placeholder="Enter course"
value={formData.course} onChange={handleChange} required />
        <input type="number" name="semester" placeholder="Enter semester"
value={formData.semester} onChange={handleChange} required />
        <input type="text" name="city" placeholder="Enter city"
value={formData.city} onChange={handleChange} required />
```

```
        <button type="submit">{editingId ? "Update Student" : "Add
Student"}</button>
      </form>

      <table>

<thead><tr><th>Name</th><th>Course</th><th>Semester</th><th>City</th><th>Actions<
/th></tr></thead>
        <tbody>
          {students.map((student) => (
            <tr key={student._id}>

<td>{student.name}</td><td>{student.course}</td><td>{student.semester}</td><td>{s
tudent.city}</td>
              <td>
                <button onClick={() => handleEdit(student)}>Edit</button>
                <button onClick={() => handleDelete(student._id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
export default App;
```

In App.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  margin: 0;
}

.container {
  width: 85%;
  margin: 30px auto;
  background-color: white;
  padding: 25px;
  border-radius: 8px;
}
```

```css
h1 {
  text-align: center;
}

form {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  margin-bottom: 25px;
}

input {
  padding: 10px;
  flex: 1;
  min-width: 160px;
}

button {
  padding: 10px 15px;
  margin: 3px;
  cursor: pointer;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  border: 1px solid #cccccc;
  padding: 10px;
  text-align: left;
}

th {
  background-color: #eeeeee;
}
```

Execution

Keep two terminals running.

Backend terminal

cd C:\MERN_Practicals\Practical-3

node server.js

Frontend terminal

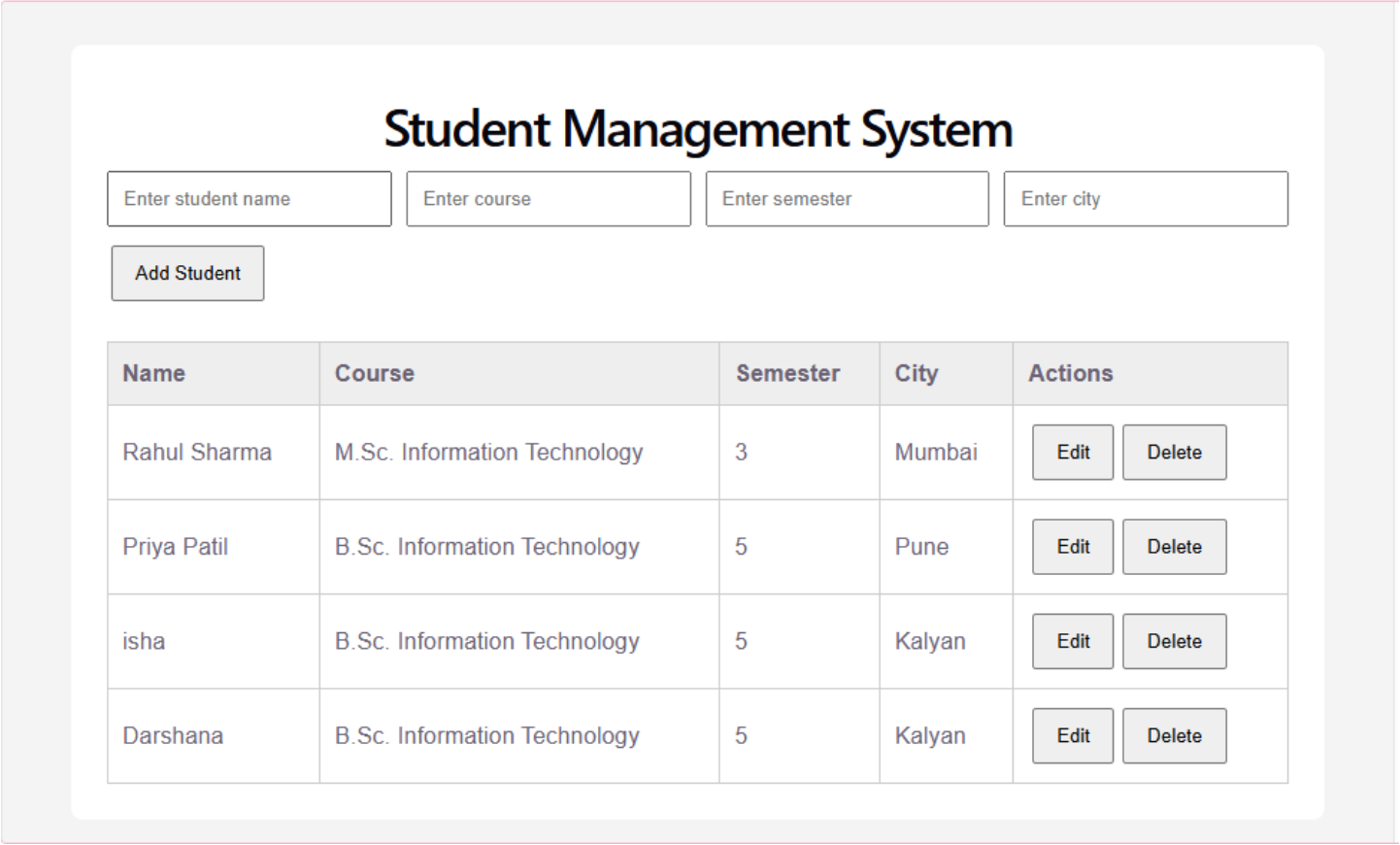
cd C:\MERN_Practicals\Practical-4\student-frontend

npm run dev

Open:

http://localhost:5173

Output



5. Complete MERN CRUD Application.

a. Design and Execute the Complete MERN Application.

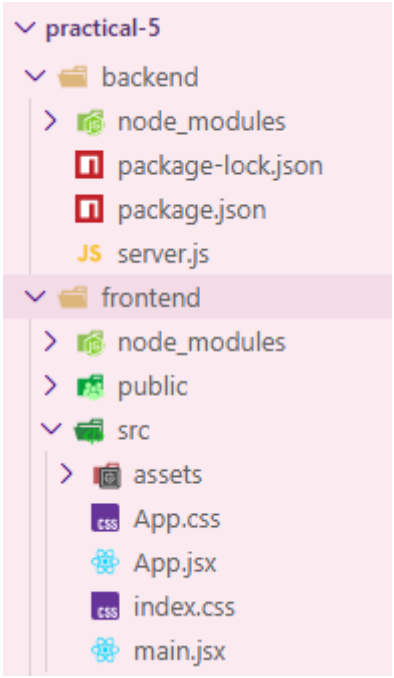
To organize and execute the complete MERN application using separate frontend and backend folders.

Steps:

Create a folder named:

Practical-5

Inside it maintain the following structure:



1. Create the Backend Folder

Open the backend folder in Visual Studio Code and initialize the Node.js project:

npm init –y

Install the required packages:

npm install express mongoose cors

2: Create server.js

Server.js

```
const express = require("express");
```

```
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
const PORT = 3000;

app.use(cors());
app.use(express.json());

mongoose
  .connect("mongodb://127.0.0.1:27017/studentDB")
  .then(() => {
    console.log("MongoDB Connected Successfully");
  })
  .catch((error) => {
    console.log("MongoDB Connection Error:", error);
  });

const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  course: {
    type: String,
    required: true,
  },
  semester: {
    type: Number,
    required: true,
  },
  city: {
    type: String,
    required: true,
  },
});

const Student = mongoose.model("Student", studentSchema);

// CREATE Student
app.post("/students", async (req, res) => {
  try {
```

```
    const student = new Student(req.body);
    const savedStudent = await student.save();
    res.status(201).json(savedStudent);
  } catch (error) {
    res.status(400).json({
      message: "Unable to add student",
      error: error.message,
    });
  }
});

// READ All Students
app.get("/students", async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (error) {
    res.status(500).json({
      message: "Unable to retrieve students",
    });
  }
});

// UPDATE Student
app.put("/students/:id", async (req, res) => {
  try {
    const updatedStudent = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      {
        new: true,
        runValidators: true,
      }
    );
    res.json(updatedStudent);
  } catch (error) {
    res.status(400).json({
      message: "Unable to update student",
      error: error.message,
    });
  }
});
```

```
// DELETE Student
app.delete("/students/:id", async (req, res) => {
  try {
    await Student.findByIdAndDelete(req.params.id);
    res.json({
      message: "Student Deleted Successfully",
    });
  } catch (error) {
    res.status(400).json({
      message: "Unable to delete student",
    });
  }
});

app.listen(PORT, () => {
  console.log(`Server Running on Port ${PORT}`);
});
```

3. Create the React Frontend

Open another terminal inside the Practical-5 folder and execute:

npm create vite@latest frontend

Select:

Framework: React

Variant: JavaScript

When asked:

Install with npm and start now?

Select:

Yes

In App.jsx

```
import { useState, useEffect } from "react";
import axios from "axios";
import "./App.css";
```

```
function App() {
  const [students, setStudents] = useState([]);
  const [formData, setFormData] = useState({
    name: "", course: "", semester: "", city: "",
  });
  const [editingId, setEditingId] = useState(null);
  const apiUrl = "http://localhost:3000/students";

  const fetchStudents = async () => {
    try {
      const res = await axios.get(apiUrl);
      setStudents(res.data);
    } catch (err) {
      console.error(err);
    }
  };

  useEffect(() => { fetchStudents(); }, []);

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      if (editingId) {
        await axios.put(`${apiUrl}/${editingId}`, formData);
        setEditingId(null);
      } else {
        await axios.post(apiUrl, formData);
      }
      setFormData({ name: "", course: "", semester: "", city: "" });
      fetchStudents();
    } catch (err) {
      console.error(err);
    }
  };

  const handleEdit = (s) => {
    setFormData({ name: s.name, course: s.course, semester: s.semester, city:
s.city });
```

```
    setEditingId(s._id);
  };

  const handleDelete = async (id) => {
    await axios.delete(`${apiUrl}/${id}`);
    fetchStudents();
  };

  return (
    <div className="main-bg">
      <header className="top-header">MERN Student Management System <span>React + Express + MongoDB</span></header>

      <div className="dashboard">
        <h2>Complete MERN Application Dashboard</h2>

        <div className="stats">
          <div className="stat-box"><p>Total Students</p><h3>{students.length}</h3></div>
          <div className="stat-box"><p>Active Semester</p><h3>5</h3></div>
          <div className="stat-box"><p>Database Status</p><h3 className="connected">Connected</h3></div>
        </div>

        <form onSubmit={handleSubmit} className="dash-form">
          <input type="text" name="name" placeholder="Enter student name" value={formData.name} onChange={handleChange} required />
          <input type="text" name="course" placeholder="Enter course" value={formData.course} onChange={handleChange} required />
          <input type="number" name="semester" placeholder="Enter semester" value={formData.semester} onChange={handleChange} required />
          <input type="text" name="city" placeholder="Enter city" value={formData.city} onChange={handleChange} required />
          <button type="submit" className="add-btn">{editingId ? "Update" : "Add Student"}</button>
        </form>

        <table>

<thead><tr><th>Name</th><th>Course</th><th>Semester</th><th>City</th><th>Actions</th></tr></thead>
          <tbody>
```

```
            {students.map((s) => (
              <tr key={s._id}>
                <td>{s.name}</td>
                <td>{s.course}</td>
                <td>{s.semester}</td>
                <td>{s.city}</td>
                <td>
                  <button className="edit-btn" onClick={() =>
handleEdit(s)}>Edit</button>
                  <button className="del-btn" onClick={() =>
handleDelete(s._id)}>Delete</button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
        <p className="footer">B.Sc. Information Technology - Full Stack
Development MERN Practice</p>
      </div>
    </div>
  );
}
export default App;
```

In App.css

```
body { margin: 0; font-family: Arial, sans-serif; background: #eef1f5; }
.main-bg { min-height: 100vh; }
.top-header {
  background: #1a1a2e;
  color: white;
  padding: 12px 25px;
  font-weight: bold;
  display: flex;
  justify-content: space-between;
}
.top-header span { font-size: 12px; color: #aaa; }

.dashboard {
  width: 90%;
  max-width: 1000px;
```

```
  background: white;
  margin: 20px auto;
  padding: 25px;
  border-radius: 10px;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
.dashboard h2 { margin-top: 0; }

.stats {
  display: flex;
  gap: 15px;
  margin: 20px 0;
}
.stat-box {
  flex: 1;
  background: #f8f9fa;
  padding: 15px;
  border-radius: 8px;
  border: 1px solid #eee;
}
.stat-box p { margin: 0; font-size: 12px; color: #666; }
.stat-box h3 { margin: 5px 0 0 0; font-size: 20px; }
.connected { color: #1a1a2e; }

.dash-form {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  margin-bottom: 20px;
}
.dash-form input {
  padding: 8px 10px;
  flex: 1;
  min-width: 130px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
.add-btn {
  background: #1a1a2e;
  color: white;
  border: none;
  padding: 8px 15px;
```

```
  border-radius: 5px;
  cursor: pointer;
}

table { width: 100%; border-collapse: collapse; }
th { background: #f1f1f1; text-align: left; padding: 10px; font-size: 13px; }
td { padding: 10px; border-bottom: 1px solid #eee; font-size: 13px; }
.edit-btn { background: #6c757d; color: white; border: none; padding: 5px 10px;
border-radius: 4px; margin-right: 5px; cursor: pointer; }
.del-btn { background: #1a1a2e; color: white; border: none; padding: 5px 10px;
border-radius: 4px; cursor: pointer; }

.footer { text-align: center; font-size: 11px; color: #888; margin-top: 20px; }
```

If the application does not start automatically, execute:

cd frontend

npm install

npm run dev

4. Install Axios

Inside the frontend folder, execute:

npm install axios

5. Run the Backend

Open a terminal inside the backend folder:

node server.js

Expected terminal output:

MongoDB Connected Successfully

Server Running on Port 3000

6**: Run the Frontend**

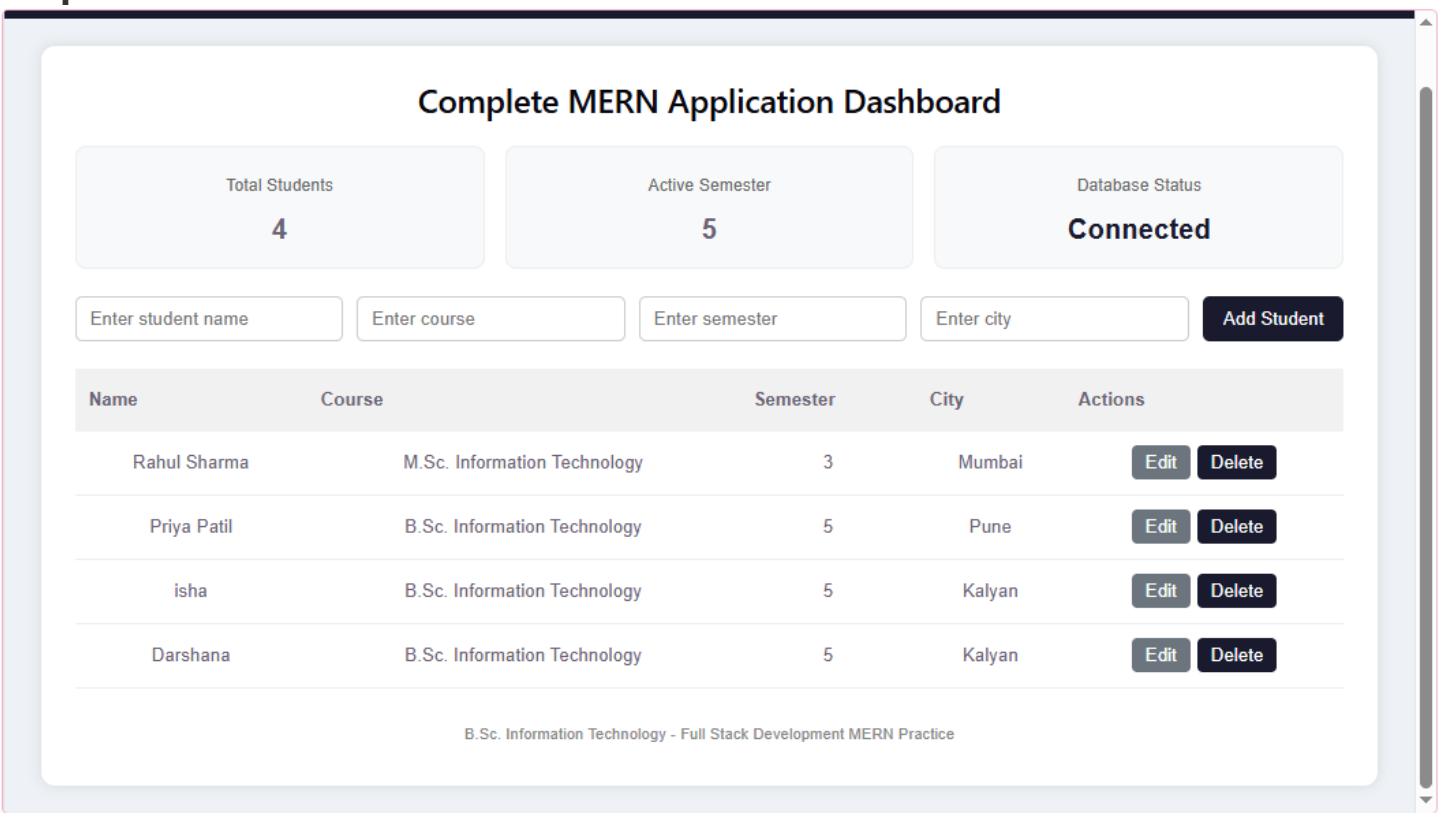
Open another terminal inside the frontend folder:

npm run dev

Open the displayed URL:

http://localhost:5175/

Output



b. Implement Complete CRUD Operations.

To perform Create, Read, Update, Delete operations through the integrated React frontend.

Src/App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const apiUrl = "http://localhost:3000/students";

  const [students, setStudents] = useState([]);
  const [formData, setFormData] = useState({
    name: "",
    course: "",
    semester: "",
    city: ""
  });

  const [editingId, setEditingId] = useState(null);
  const [message, setMessage] = useState("");

  const fetchStudents = async () => {
    try {
      const response = await axios.get(apiUrl);
      setStudents(response.data);
    } catch (error) {
      setMessage("Unable to retrieve student records.");
      console.error(error);
    }
  };

  useEffect(() => {
    fetchStudents();
  }, []);

  const handleChange = (event) => {
    setFormData({
      ...formData,
```

```
      [event.target.name]: event.target.value
    });
  };

  const clearForm = () => {
    setFormData({
      name: "",
      course: "",
      semester: "",
      city: ""
    });
    setEditingId(null);
  };

  const handleSubmit = async (event) => {
    event.preventDefault();
    try {
      if (editingId) {
        await axios.put(
          `${apiUrl}/${editingId}`,
          formData
        );
        setMessage("Student record updated successfully.");
      } else {
        await axios.post(apiUrl, formData);
        setMessage("Student record added successfully.");
      }
      clearForm();
      fetchStudents();
    } catch (error) {
      setMessage("Unable to save the student record.");
      console.error(error);
    }
  };

  const handleEdit = (student) => {
    setFormData({
      name: student.name,
      course: student.course,
      semester: student.semester,
      city: student.city
    });
```

```
    setEditingId(student._id);
    setMessage("Edit the details and click Update Student.");
  };

  const handleDelete = async (id) => {
    try {
      await axios.delete(`${apiUrl}/${id}`);
      setMessage("Student record deleted successfully.");
      fetchStudents();
    } catch (error) {
      setMessage("Unable to delete the student record.");
      console.error(error);
    }
  };

  return (
    <div className="container">
      <h1>Student Management System</h1>

      {message && (
        <p className="message">{message}</p>
      )}

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          name="name"
          placeholder="Enter student name"
          value={formData.name}
          onChange={handleChange}
          required
        />
        <input
          type="text"
          name="course"
          placeholder="Enter course"
          value={formData.course}
          onChange={handleChange}
          required
        />
        <input
          type="number"
```

```
          name="semester"
          placeholder="Enter semester"
          value={formData.semester}
          onChange={handleChange}
          min="1"
          required
        />
        <input
          type="text"
          name="city"
          placeholder="Enter city"
          value={formData.city}
          onChange={handleChange}
          required
        />

        <button type="submit">
          {editingId ? "Update Student" : "Add Student"}
        </button>

        {editingId && (
          <button
            type="button"
            onClick={clearForm}
          >
            Cancel
          </button>
        )}
      </form>

      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Course</th>
            <th>Semester</th>
            <th>City</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {students.length === 0 ? (
```

```
            <tr>
              <td colSpan="5">
                No student records available.
              </td>
            </tr>
          ) : (
            students.map((student) => (
              <tr key={student._id}>
                <td>{student.name}</td>
                <td>{student.course}</td>
                <td>{student.semester}</td>
                <td>{student.city}</td>
                <td>
                  <button
                    type="button"
                    onClick={() => handleEdit(student)}
                  >
                    Edit
                  </button>
                  <button
                    type="button"
                    onClick={() =>
                      handleDelete(student._id)
                    }
                  >
                    Delete
                  </button>
                </td>
              </tr>
            ))
          )}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Src/App.css

```css
body {

  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f3f5f8;
}

.container {
  width: 88%;
  margin: 35px auto;
  padding: 25px;
  background-color: white;
  border-radius: 10px;
}

h1 {
  text-align: center;
  color: #1f2937;
}

.message {
  padding: 12px;
  background-color: #e8f5e9;
  color: #176b2c;
  border-radius: 6px;
  text-align: center;
}

form {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin: 25px 0;
}

input {
  flex: 1;
  min-width: 160px;
  padding: 11px;
  border: 1px solid #cccccc;
  border-radius: 6px;
}
```

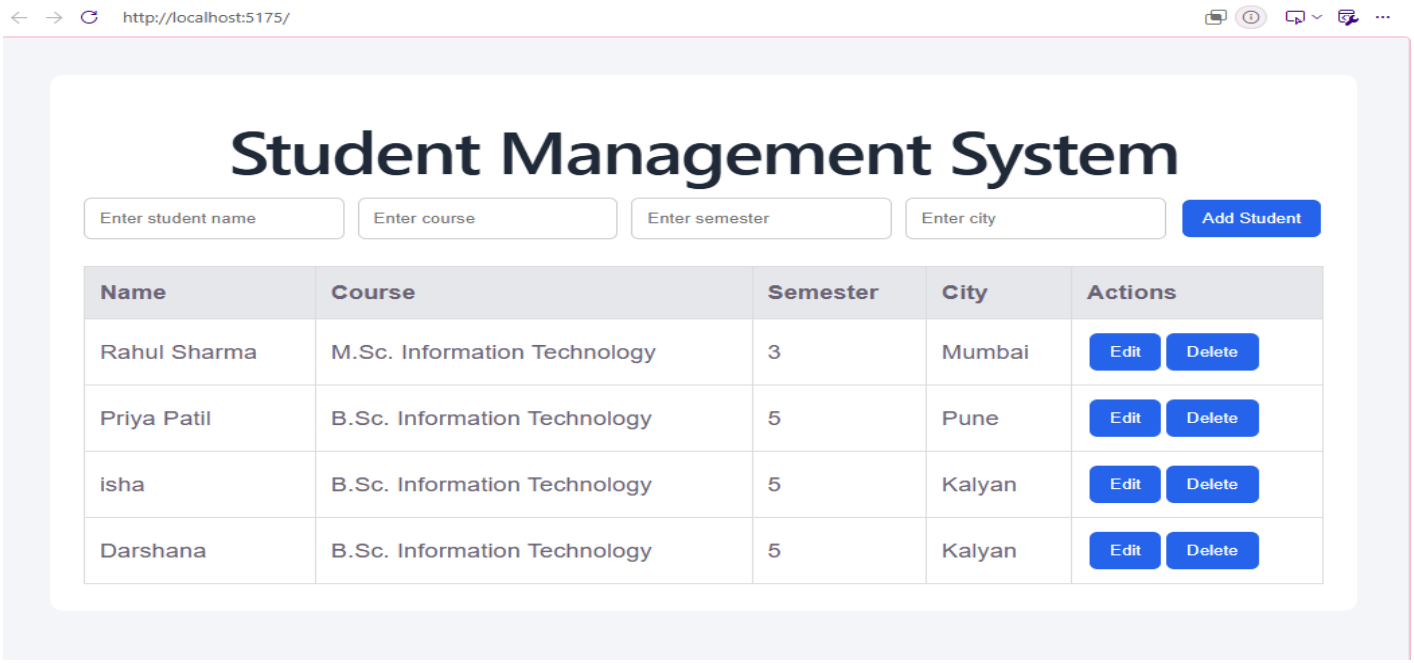
```css
button {
  padding: 10px 15px;
  margin: 2px;
  border: none;
  border-radius: 6px;
  background-color: #2563eb;
  color: white;
  cursor: pointer;
}

button:hover {
  opacity: 0.85;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  border: 1px solid #dddddd;
  padding: 11px;
  text-align: left;
}

th {
  background-color: #e5e7eb;
}
```


http://localhost:5175/
Student Management System
Enter student name
Enter course
Enter semester
Enter city
Add Student
Name
Course
Semester
City
Actions
Rahul Sharma
M.Sc. Information Technology
3
Mumbai
Edit
Delete
Priya Patil
B.Sc. Information Technology
5
Pune
Edit
Delete
isha
B.Sc. Information Technology
5
Kalyan
Edit
Delete
Darshana
B.Sc. Information Technology
5
Kalyan
Edit
Delete

C. Test and Validate the Complete MERN Workflow.

Testing Procedure

Test 1: Add a Student Record

Enter:

Name: Darshana.

Course: B.Sc. Information Technology

Semester: 5

City: Kalyan

Click:

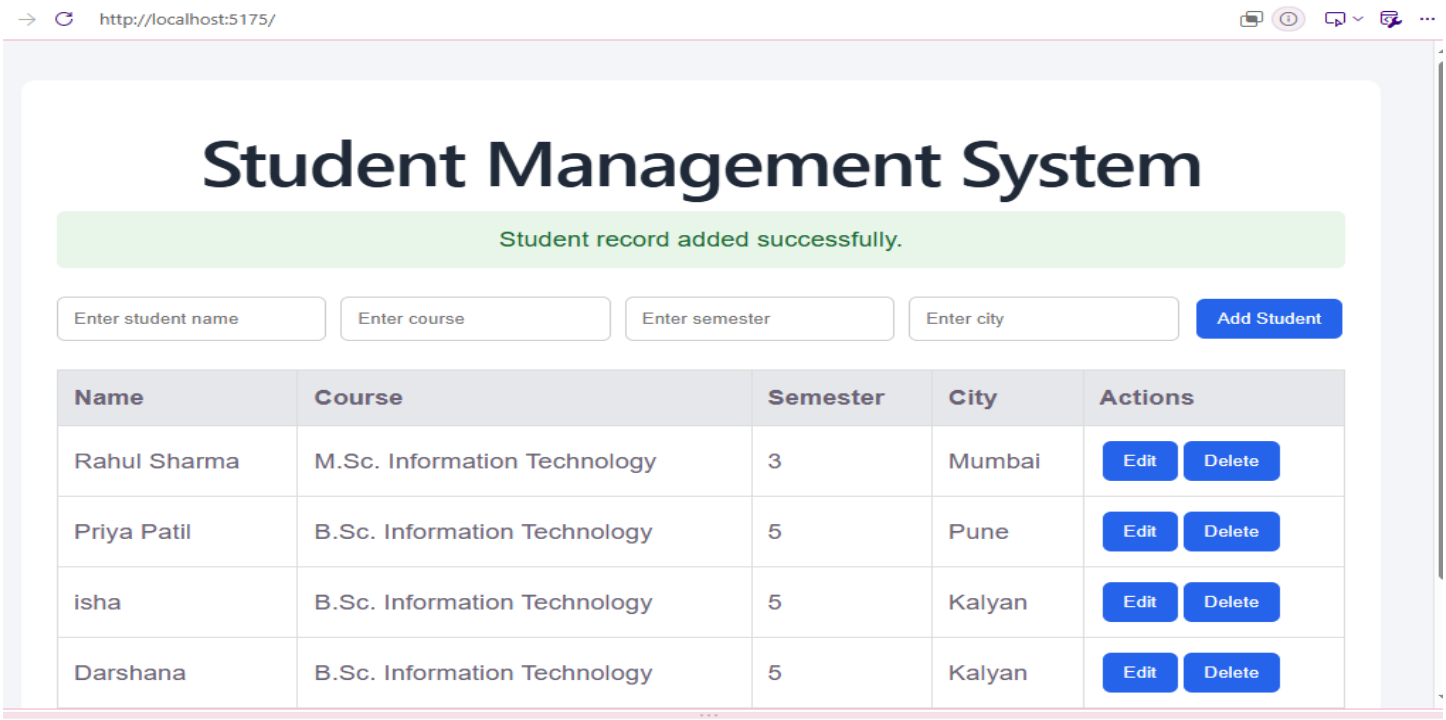
Add Student

The record should appear in the table and should also be stored in MongoDB.

Test 2: Retrieve Records

Refresh the browser.

The previously inserted records should remain visible, confirming that the data is stored in MongoDB.



Test 3: Update a Student Record

Click Edit.

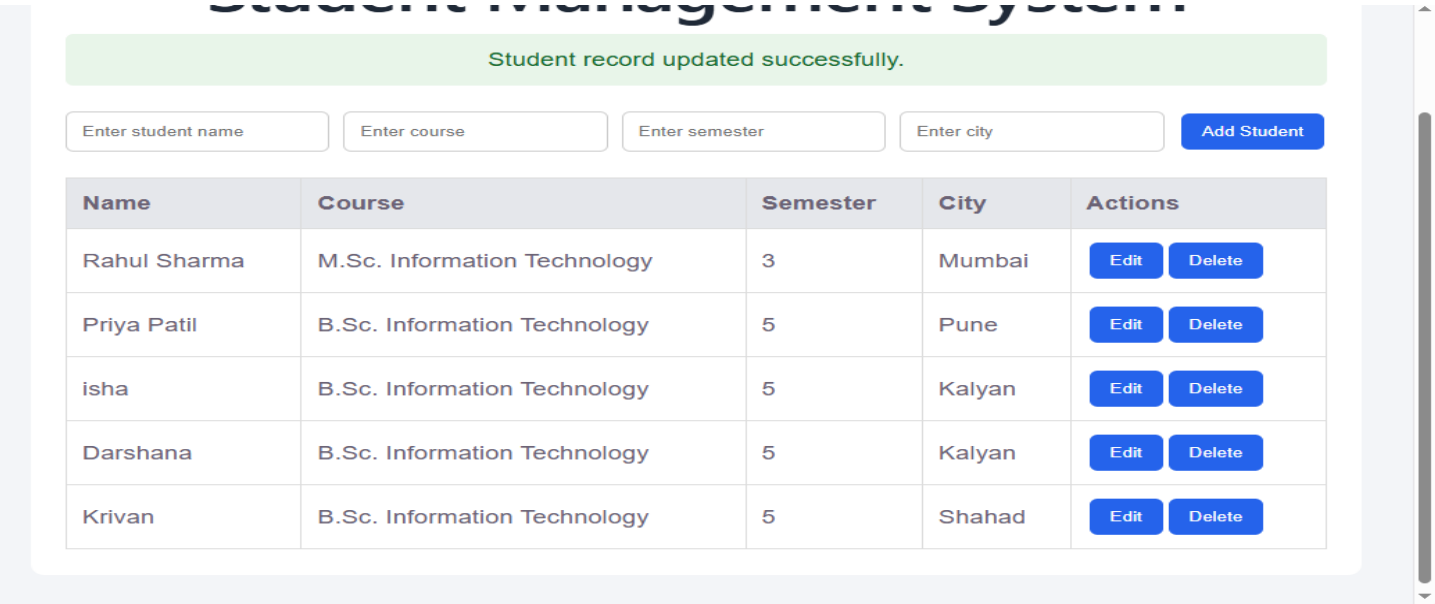
Modify the required field, such as:

Semester: 6

Click:

Update Student

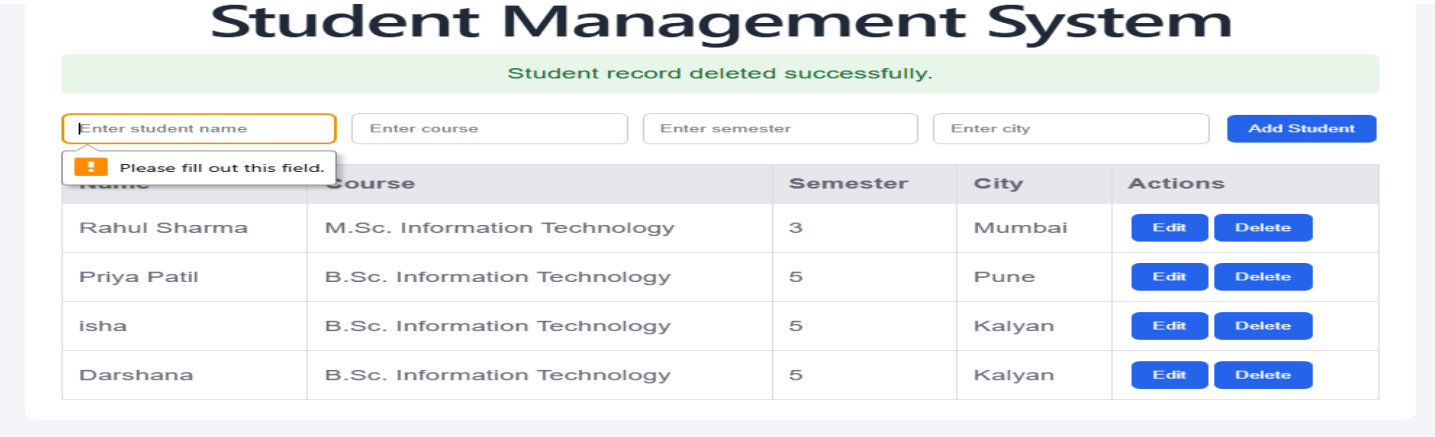
The updated value should appear in the table.



Test 4: Delete a Student Record

Click Delete beside a student record.

The selected record should be removed from the table and MongoDB.



Test 5: Validate Required Fields

Try to submit the form without entering the student name.

The browser should prevent submission because the field is marked as required.

Output

